



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

Timotej Kuzma

RAZVOJ RAČUNALNIŠKE IGRE Z INTELIGENTNIMI AKTERJI

Diplomsko delo

Maribor, avgust 2024



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

Timotej Kuzma

RAZVOJ RAČUNALNIŠKE IGRE Z INTELIGENTNIMI AKTERJI

Diplomsko delo

Maribor, avgust 2024

RAZVOJ RAČUNALNIŠKE IGRE Z INTELIGENTNIMI AKTERJI

Diplomsko delo

Študent:	Timotej Kuzma
Študijski program:	visokošolski strokovni študijski program Računalništvo in informacijske tehnologije
Mentor:	doc. dr. Marko Bizjak, mag. rač. in inf. tehnol.
Somentor:	doc. dr. Štefan Kohek, mag. rač. in inf. tehnol.

ZAHVALA

Zahvaljujem se mentorju, doc. dr. Marku Bizjaku
in somentorju, doc. dr. Štefanu Koheku, ker sta me
usmerjala med pisanjem diplomskega dela.
Zahvaljujem se tudi vsem zaposlenim pri Epic Games,
ker so s svojim izjemnim delom razvoj iger približali
vsakomur.

Razvoj računalniške igre z inteligentnimi akterji

Ključne besede: Unreal Engine, proceduralno ustvarjanje vsebine, inteligentni akter, ciljno usmerjeno načrtovanje akcij

UDK: 004.96(043.2)

Povzetek

V diplomskem delu predstavimo postopek izdelave tretjeosebne srednjeveške računalniške igre v pogonu Unreal Engine. Podrobno opišemo ozadje razvoja računalniških iger, proceduralno gradnjo terena in razvoj inteligentnih akterjev v okvirju principa delovanja igre, ki temeljijo na algoritmu GOAP. Na koncu analiziramo delovanje razvite igre na podlagi meritev števila sličic na sekundo glede na število akterjev, hitrosti izvajanja ključnih odsekov kode in kvalitete izrisa.

Development of a Computer Game with Intelligent Actors

Keywords: Unreal Engine, Procedural Content Generation, intelligent actor, Goal-Oriented Action Planning

UDC: 004.96(043.2)

Abstract

In the thesis, we present the creation process of a third-person medieval computer game in Unreal Engine. We describe in detail the background of the development of computer games, the procedural construction of terrain and the development of intelligent actors within the framework, based on the GOAP algorithm. Finally, we analyze the performance of the developed game based on measurements of the number of frames per second in relation to the number of actors, the execution speed of key sections of the code and the quality of rendering.



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko
Koroška cesta 46
2000 Maribor, Slovenija



IZJAVA O AVTORSTVU ZAKLJUČNEGA DELA

Ime in priimek študent-a/-ke: Timotej Kuzma
Študijski program: RAČUNALNIŠTVO IN INFORMACIJSKE TEHNOLOGIJE
Naslov zaključnega dela: Razvoj računalniške igre z inteligentnimi akterji

Mentor: doc. dr. Marko Bizjak, mag. rač. in inf. tehnol.

Somentor: doc. dr. Štefan Kohek, mag. rač. in inf. tehnol.

Podpisan-i/-a študent/-ka Timotej Kuzma

- izjavljam, da je zaključno delo rezultat mojega samostojnega dela, ki sem ga izdelal/-a ob pomoči mentor-ja/-ice oz. somentor-ja/-ice;
- izjavljam, da sem pridobil/-a vsa potrebna soglasja za uporabo podatkov in avtorskih del v zaključnem delu in jih v zaključnem delu jasno in ustrezno označil/-a;
- na Univerzo v Mariboru neodplačno, neizključno, prostorsko in časovno neomejeno prenašam pravico shranitve avtorskega dela v elektronski obliki, pravico reproduciranja ter pravico ponuditi zaključno delo javnosti na svetovnem spletu preko DKUM; sem seznanjen/-a, da bodo dela deponirana/objavljena v DKUM dostopna široki javnosti pod pogoji licence Creative Commons BY-NC-ND, kar vključuje tudi avtomatizirano indeksiranje preko spleta in obdelavo besedil za potrebe tekstovnega in podatkovnega rudarjenja in ekstrakcije znanja iz vsebin; uporabnikom se dovoli reproduciranje brez predelave avtorskega dela, distribuiranje, dajanje v najem in priobčitev javnosti samega izvirnega avtorskega dela, in sicer pod pogojem, da navedejo avtorja in da ne gre za komercialno uporabo;
- dovoljujem objavo svojih osebnih podatkov, ki so navedeni v zaključnem delu in tej izjavi, skupaj z objavo zaključnega dela.

Uveljavljam permisivnejšo obliko licence Creative Commons: (navedite obliko)

Kraj in datum:

Maribor, 7. 8. 2024

Podpis študent-a/-ke:

KAZALO VSEBINE

1	Uvod	1
2	Ozadje računalniških iger.....	2
2.1	Igralni pogoni	2
2.2	Inteligentni nasprotniki.....	3
2.2.1	Pogosti pristopi razvoja	4
2.2.2	Ciljno usmerjeno načrtovanje akcij	6
3	Razvoj računalniške igre	8
3.1	Uporabljena orodja.....	8
3.1.1	Unreal Engine 5.3	8
3.1.2	Blender 3.6	10
3.2	Vzpostavitev projekta v Unreal Engine.....	10
3.3	Ustvarjanje lastnih 3D modelov.....	12
3.4	Proceduralno ustvarjanje vsebine	15
3.4.1	Osnove PCG v programu Unreal Engine.....	16
3.4.2	Uporaba PCG v naši igri	16
3.5	Inteligentni akterji v okviru principa delovanja igre	23
3.5.1	Implementacija algoritma GOAP	26
3.5.1.1	Pomožni razredi	30
3.5.1.2	Inicializacija	35
3.5.1.3	Dijkstrin algoritem.....	38
3.5.1.4	Prednostna vrsta	43
3.5.2	Uvoz animacij.....	47
3.5.3	Določanje poteka igre z Blueprint	48
3.5.3.1	Streljanje z lokom.....	50
3.5.3.2	Uporaba algoritma GOAP v razredu Blueprint.....	57
4	Rezultati	62
5	Sklep.....	67

KAZALO SLIK

Slika 3.1: Ustvarjanje projekta	11
Slika 3.2: Urejevalnik nivoja	12
Slika 3.3: Izdelana miza v notranjosti polzemljanke	13
Slika 3.4: Izdelana notranjost polzemljanke	13
Slika 3.5: Oblikovana notranjost hleva	13
Slika 3.6: Izdelan lok v programu Blender	14
Slika 3.7: Izdelan meč v programu Blender	14
Slika 3.8: Izdelan ščit v programu Blender	15
Slika 3.9: Rezultat našega grafa PCG	17
Slika 3.10: Pretvarjanje modela v razred Blueprint	18
Slika 3.11: Vpliv krivulj na generiranje terena	18
Slika 3.12: Uporaba vozlišča Projection	19
Slika 3.13: Vozlišče Projection prestavi vse točke iz notranjosti krivulje na tla	20
Slika 3.14: Z vozliščem Distance točkam iz vozlišča Projection nastavimo gostoto glede na oddaljenost od krivulj obzidja in poti	21
Slika 3.15: Vozlišče Distance točkam v bližini krivulj priredi nižjo, bolj oddaljenim pa višjo gostoto	21
Slika 3.16: Uporaba vozlišč Spawn Actor in Transform Points	22
Slika 3.17: Z vozliščema Spawn Actor postavljene zgradbe	23
Slika 3.18: Prizor iz igre, ko se akterji združujejo pred vrati obzidja	23
Slika 3.19: Branitelj (na levi strani, z lokom v roki) pokonča oblegovalca (desno)	24
Slika 3.20: Vrata obzidja padejo, akter (desno) zasleduje igralca (levo)	24
Slika 3.21: Prizor iz igre, ko akter (v sredini) udari igralca (na levi)	25
Slika 3.22: Konec igre, ker je igralcu zmanjkalo življenja	26
Slika 3.23: Privzeto ustvarjen Unrealov razred v programskem jeziku C++	27
Slika 3.24: Metode C++, dosegljive iz razreda Blueprint	27
Slika 3.25: Uporaba metode C++ v razredu Blueprint	28
Slika 3.26: Iskanje metod v urejevalniku Blueprint	28
Slika 3.27: Deklaracija dogodkov Blueprint	29

Slika 3.28: Implementacija dogodka GOAPBreakDoorEvent.....	29
Slika 3.29: Deklaracije spremenljivk, dosegljivih iz razreda Blueprint.....	30
Slika 3.30: Spreminjanje vrednosti spremenljivke GOAPBowAttackRange.....	30
Slika 3.31: Ilustracija delovanja akcij pri GOAP.....	32
Slika 3.32: Struktura WorldState	32
Slika 3.33: Razred Action in izpeljan razred ActionEquipSword	33
Slika 3.34: Razred ActionMoveToTheHighGround	34
Slika 3.35: Struktura WorldStateGraphConnection in razred WorldStateGraphNode ..	35
Slika 3.36: Metoda fillActionsList.....	36
Slika 3.37: Metoda fillAdjacencyList	37
Slika 3.38: Metoda FindWorldStateStartpointsForActions	38
Slika 3.39: Metoda prepareDijkstra	39
Slika 3.40: Metoda performDijkstra.....	41
Slika 3.41: Metoda outputBestPath.....	42
Slika 3.42 Izpis načrta GOAP, prva akcija je na dnu, zadnja pa na vrhu izpisa	42
Slika 3.43: Razred PriorityQueue	44
Slika 3.44: Metoda fixHeapAtIndex	45
Slika 3.45: Metode heapIsEmpty, getMinDistanceNode in insertNewNode	46
Slika 3.46: Metoda deleteRootNode	47
Slika 3.47: Razred BP_Soldier1	49
Slika 3.48: Začetek grafa iz vozlišča RightMouseButton.....	50
Slika 3.49: Uporaba vozlišč PlayMontage	51
Slika 3.50: Uporaba vozlišč StopAnimMontage	52
Slika 3.51: Dogodek ob pritisku levega klika.....	52
Slika 3.52: Nadaljevanje grafa, ko je SwordEquipped = false	53
Slika 3.53: Uporaba vozlišča SpawnActorFromClass	53
Slika 3.54: Uporaba vozlišča LineTraceForObjects	54
Slika 3.55: Klic funkcije Initialize iz razreda BP_Arrow_Projectile	55
Slika 3.56: Definicija metode Initialize v razredu BP_ArrowProjectile	55
Slika 3.57: Uporaba vozlišča SuggestProjectileVelocityCustomArc.....	56

Slika 3.58: Klic dogodka TakeTheDamage.....	57
Slika 3.59: Definicija dogodka TakeTheDamage	57
Slika 3.60: Začetno vozlišče funkcije SetGOAPActionCosts	58
Slika 3.61: Nastavljanje spremenljivke GOAPDoorToPlayerDistance.....	58
Slika 3.62: Klic metode GOAPExecuteDijkstra	59
Slika 3.63: Določanje vrednosti vhodnih parametrov	59
Slika 3.64: Navigacijska mreža in instance razreda BP_HighGroundPosition.....	60
Slika 3.65: Definicija dogodka GOAPMoveToHighGroundEvent	61

KAZALO GRAFOV

Grafikon 1: Hitrost izvedbe GOAPExecuteDijkstra v odvisnosti od števila akterjev.....	64
Grafikon 2: FPS v odvisnosti od števila akterjev	65
Grafikon 3: FPS v odvisnosti od kvalitete grafičnega izrisa.....	66

UPORABLJENI SIMBOLI IN KRATICE

GOAP – ciljno usmerjeno načrtovanje akcij (angl. *Goal-Oriented Action Planning*)

AI – umetna inteligenca (angl. *Artificial Intelligence*)

PCG – proceduralno ustvarjanje vsebine (angl. *Procedural Content Generation*)

3D – tridimenzionalen (angl. *Three-dimensional*)

FPS – število sličic na sekundo (angl. *Frames per Second*)

1 UVOD

Računalniške igre so priljubljen način preživljanja prostega časa. S čedalje večjim prodiranjem digitalne tehnologije v vsakdanje življenje posameznika so tudi računalniške igre postale del vsakdana. Na začetku so bile namenjene predvsem tehničnim navdušencem in igranju na igralnih avtomatih, danes pa so na voljo za najrazličnejše ciljne skupine. Zato je industrija računalniških iger postala ena izmed vodilnih na področju zabave. Po podatkih revije Forbes iz leta 2022 letno prinese 184,4 milijarde ameriških dolarjev profita [1]. Za primerjavo, glasbena in filmska industrija prineseta le skromnih 26 milijard ameriških dolarjev [1]. S povečevanjem občinstva in razvojem tehnologije so se razvijale tudi igre. Danes imamo igre za najrazličnejša občinstva, vanje pa se vlaga tudi veliko denarja. Tako imamo na voljo igre namenjene učenju, simulacijske igre, igre za mobilne telefone, večigralske igre in še veliko več.

Cilj našega diplomskega dela je razviti tretjeosebno enoigralsko računalniško igro s proceduralno zgrajenim terenom in inteligentnimi akterji z uporabo orodij Unreal Engine in Blender. Akterji bodo razdeljeni v dve skupini. Prva skupina bo skušala zavzeti obzidje, druga pa pomagati igralcu pri obrambi. Inteligentni akterji morajo za igralca predstavljati izziv, prav tako pa se morajo obnašati čim bolj realistično in početi to, kar je v trenutni fazi obleganja smiselno, seveda pa mora igra ostati čim bolj zabavna za igralca. Projekt naj bo zasnovan tako, da bo omogočal hitro nadgrajevanje in čim večjo možnost ponovne uporabe v drugih projektih.

V prvem delu diplomskega dela bomo predstavili kratko zgodovino začetkov računalniških iger, igralnih pogonov in pogoste pristope razvoja inteligentnih akterjev, v drugem delu pa orodja, ki smo jih pri razvoju uporabljali, razvoj proceduralno zgrajenega terena in razvoj inteligentnega akterja z uporabo ciljno usmerjenega načrtovanja akcij (angl. *Goal-Oriented Action Planning*, *GOAP*). Pri rezultatih bomo predstavili zmogljivost naše implementacije igre v odvisnosti od kompleksnosti izvedbe.

2 OZADJE RAČUNALNIŠKIH IGER

Računalniške igre so računalniški programi, ki igralcu omogočajo interaktivno spreminjanje virtualnega sveta. Za igranje se uporabljajo vhodne naprave, kot so miška, tipkovnica ali igralni plošček. Računalnik poteze igralca obdela in pošlje na izhodno napravo, na primer monitor in zvočnik. Navadno igralec nadzoruje nek lik v igri, ki ga premika po virtualnem svetu in svet spreminja. Računalniške igre morajo biti oblikovane čim bolj privlačno, da zagotovijo celostno avdiovizualno izkušnjo. Lahko so enoigralske, kjer uporabnik spreminja svet, ki je navadno poseljen z liki, ki jih upravlja računalnik. Poznamo pa tudi večigralske igre, kjer več igralcev spreminja isti svet. To se lahko doseže z razdelitvijo zaslona, prek interneta ali pa kako drugače.

Na začetku so računalniške igre razvijali predvsem znanstveniki v raziskovalne namene, z razvojem igralnih konzol in osebnih računalnikov pa se je njihov doseg močno povečal. Zgodnji primer računalniške igre je OXO, računalniška verzija križcev in krožcev, ki jo je leta 1952 ustvaril britanski profesor A.S. Douglas, leta 1958 pa je William Higinbotham ustvaril igro Tennis for Two na analognem računalniku. Naslednji pomembni mejnik je igra Pong, poznana kot prva arkadna igra. Izdal jo je Atari leta 1972, tri leta kasneje pa je izšla različica za domačo uporabo. V sedemdesetih letih dvajsetega stoletja so izšle tudi igre Space Invaders, Pac-Man, Donkey Kong in Microsoft Flight Simulator [2]. V začetku osemdesetih let prejšnjega stoletja je na konzolnih igrah temelječa ameriška industrija videoiger doživela velik padec iz več razlogov, predvsem zaradi pojava konkurenčnih iger, namenjenih igranju na osebni računalnik. Industrija se je pobrala s prihodom konzole Nintendo Entertainment System, ki je izboljšala uporabniško izkušnjo igranja na konzolah. Nintendo je v tistem času razvil veliko še danes prepoznavnih franšiz, kot sta Super Mario Bros in The Legend of Zelda [2]. V devetdesetih letih so izšle številne igre, ki so priljubljene še danes.

2.1 Igralni pogoni

Z naraščanjem priljubljenosti računalniških iger se je pojavila potreba po možnosti ponovne uporabe kode. Ker je bilo navadno treba igro razviti iz nič, so se pojavili igralni

pogoni (angl. *Game Engines*), ki so razvijalcem tudi omogočili, da s samo eno programsko opremo razvijejo več iger za več platform.

Prvi igralni pogon je leta 1991 razvil id Software za franšizo Commander Keen, z imenom Keen Engine. Menili so, da je razvoj igralnega pogona bolj pomemben kot razvoj igre same zaradi vseh možnosti, ki jih ponuja. Širši javnosti so svoj pogon (z novim imenom Doom Engine) predstavili leta 1993 ob izidu prvoosebne strelske igre Doom. Igra je bila zelo uspešna, z njo pa je uspeh delil Doom Engine, ki je igro poganjal. Licenco za njegovo uporabo so pridobila številna podjetja, razvijalcem pa je med drugim omogočal, da so v igro dodali lastne teksture, like, orožja in nivoje. Igre je tako poganjal pogon podjetja id Software, gradnike pa so izdelali razvijalci sami. Doom Engine je uspel ustvariti iluzijo tridimenzionalnega (angl. *Three-dimensional*, 3D) sveta, z izrisovanjem le vidnega dela sveta in uporabo tehnike, imenovane binarna razdelitev prostora, pa je kljub nizki zmogljivosti takratnih računalnikov omogočal dobro odzivnost [3].

Tri leta kasneje je id Software za igro Quake razvil Quake Engine, ki je dejansko izrisoval 3D prostor in uvedel dinamično senčenje pri premikajočih predmetih ter statične svetlobne karte (angl. *lightmap*) pri statičnih objektih. Leta 1996 je prav tako izšel pogon Unreal Engine, ki ga je razvil Tim Sweeney za igro Unreal. Njegovo podjetje Epic Games je igro izdalo leta 1998. Pogon je še zmeraj standard v industriji računalniških iger, poleg razvoja visoko kakovostnih produktov pa omogoča izdelavo iger tudi neodvisnim razvijalcem. Njegov največji tekmeč je leta 2005 izdan pogon Unity [3].

2.2 Inteligentni nasprotniki

Skupaj z računalniškimi igrami so se pojavljali vedno novi pristopi razvoja inteligentnega nasprotnika. Želimo si, da bi se računalniško nadzorovani liki obnašali čimbolj realistično, da bi predstavljali izziv za igralca. To je zelo dobro uspelo podjetju DeepMind Technologies, ki je s svojim programom AlphaGo z uporabo strojnega učenja leta 2016 uspel premagati Leeja Sedola, enega najboljših igralcev v igri Go z

rezultat 4 proti 1 [4]. Takšnega nasprotnika si navadno ne želimo, saj mora imeti dovolj pomanjkljivosti, da jih lahko naša ciljna skupina igralcev premaga.

2.2.1 Pogosti pristopi razvoja

Med najbolj primitivnimi pristopi so statično vgrajeni (angl. *hardcoded*) [5] pogojni stavki. Neposredno v kodi napišemo, kaj naj se izvede in kakšni pogoji so za izvedbo potrebni. Pri igri Pong, kjer proti računalniku igramo namizni tenis, bi bilo najbolj enostavno, da računalnik postavi lopar na višino žoge. To lahko preprosto realiziramo s pogojnimi stavki: če je lopar na ekranu pod žogo, ga mora inteligentni nasprotnik premakniti navzgor, v nasprotnem primeru pa navzdol [5].

Primer je ekvivalenten odločitvenemu drevesu (angl. *Decision Tree*) [6], pristopu, kjer se v vsakem vozlišču drevesa preverja nek pogoj. Vsako vozlišče ima dva otroka, do katerih pridemo, če je pogoj v staršu izpolnjen oziroma ni izpolnjen. Tako se od korena drevesa premikamo proti listom, kjer definiramo neko akcijo, ki se mora ob takih pogojih izvesti. Na odločitvena drevesa lahko torej gledamo kot na gnezdene pogojne stavke [6].

Med naprednejše pristope sodijo končni avtomati (angl. *Finite State Machines*) [7]. Gre za sistem z možnimi stanji, v katerih se lahko nahaja akter. Stanji sta med seboj lahko povezani enosmerno ali dvosmerno. Za prehod med stanji mora biti izpolnjen pogoj, ki je definiran pri vsaki povezavi posebej. Na primer, če je agent v stanju patroljiranja, lahko ob zaznavi igralca preide v stanje sledenja igralcu, od koder lahko ob dovolj majhni razdalji preide v stanje napadanja. Če v stanju napadanja oziroma sledenja igralca ne vidi več, se vrne v stanje patroljiranja [7].

Opisani pristop nam sicer omogoča intuitivno ustvarjanje dokaj kompleksnega vedenja, vendar ima očitno slabost, razvidno iz prejšnjega primera: prehodi med stanji so vezani na trenutno stanje agenta, zato moramo prehod nazaj v stanje patroljiranja definirati tako za stanje napadanja kot tudi za stanje sledenja. Z dobro načrtovanim končnim avtomatom se lahko takšnemu podvajanju prehodov v veliki meri izognemo,

vendar se vselej lahko zgodi, da želimo preiti v neko stanje ne glede na stanje, v katerem se nahajamo. Na primer, če želimo, da agent preide v stanje napadanja vedno, ko je dovolj blizu igralca. Pri tem si lahko pomagamo z vedenjskimi drevesi (angl. *Behavior Trees*) [5], ki so skorajda postala standardni pristop ustvarjanja AI v igrah prejšnjega desetletja [8]. Čeprav so implementacije vedenjskih dreves različne je njihova osnovna struktura skoraj enaka. V osnovi so dokaj podobna odločitvenim drevesom, vendar med pristopoma obstajajo pomembne razlike. Vozlišča vedenjskih dreves navadno vsebujejo funkcijo, ki lahko vrača 3 vrednosti: »uspelo«, »ni uspelo« in »se izvaja«, na podlagi katerih se algoritem spusti po veji navzdol, počaka pri vozlišču ali pa se premakne na sosednje vozlišče. Namesto vozlišč, v katerih pri vedenjskih drevesih izbiramo med dvema možnostma, imamo tako imenovane dekoratorje (angl. *decorators*), iz katerih izhaja samo en otrok. Pogoji se torej pri vedenjskih drevesih preverjajo v dekoratorjih. Če se v listu veje, ki izhaja iz dekoratorja, predpisana akcija še vedno izvaja, dekoratorjeva funkcija vrne »se izvaja«. Funkcije listov drevesa, ki se izvajajo, vračajo vrednost »se izvaja«. Večina implementacij pozna tudi dekoratorje, v katerih se pogoji ne preverjajo. Primer je dekorator »ponavlja«, ki otroka ponavlja, dokler otrokova funkcija ne vrne »akcija ni uspela«. Vedenjska drevesa so močno orodje za izdelavo prepričljivega agenta v igri, vendar lahko delo z njimi postane precej težavno [5]. Kljub temu se je pristop precej uveljavil, grafičen urejevalnik vedenjskih dreves nam med drugim ponuja tudi Unreal Engine [9].

Pri nekaterih igrah bi radi namesto nabora odločitev, ki jih mora akter sprejeti, imeli nabor akcij, ki jih lahko v danem trenutku izvede na podlagi uporabnosti posamezne akcije v danem trenutku. V tem primeru si lahko pomagamo s sistemi uporabnosti (angl. *Utility Systems*) [6], kjer agent izbira med akcijami, ki so mu na voljo in za vsako akcijo izračuna njeno funkcijo uporabnosti. Ta vrača število, ki agentu pove, kako uporabna je akcija za doseg njegovega cilja. Agent nato med vsemi razpoložljivimi akcijami izbere tisto, ki je zanj najbolj ugodna. Izvedba akcije agenta privede v novo stanje. Pristop je torej podoben končnim avtomatom, le da se tu s primerjanjem rezultatov funkcij uporabnosti izbere najbolj ugodno povezavo med stanji [6].

2.2.2 Ciljno usmerjeno načrtovanje akcij

Med različicami sistemov na podlagi uporabnosti je tudi ciljno usmerjeno načrtovanje akcij (angl. *Goal-Oriented Action Planning, GOAP*) [10], ki smo ga pri ustvarjanju inteligentnega nasprotnika uporabili pri naši igri. Razvili so ga za igro F.E.A.R. (2005), ker jim je ustvarjanje inteligentnih akterjev s končnimi avtomati povzročalo težave. Če so želeli dodati razmeroma preprosto vedenje, so morali popraviti povezave med velikim številom stanj v končnem avtomatu, kar je močno otežilo razvoj kompleksnega vedenja. Pri razvoju algoritma so se zgledovali po algoritmu STRIPS [10].

Gre za naprednejšo obliko ciljno usmerjenega načrtovanja akcij, saj GOAP namesto preproste izbire najboljše akcije iz trenutnega stanja poišče najbolj ugodno zaporedje akcij, ki nas bo iz trenutnega stanja privedlo do izbranega cilja. Pristop nam prihrani ustvarjanje zapletenega končnega avtomata, vedenje agentov pa je dovolj kompleksno, da igralcu delujejo inteligentno. Prav tako je ob ustrezni implementaciji algoritem zelo razširljiv [6].

Pri tem algoritmu je vedenje akterja odvisno od njegovega stanja in od stanja sveta. Zamislimo si srednjeveško igro, v kateri imamo oblegovalce in branitelje obzidja. Če je akter oblegovalec opremljen z mečem in je njegov cilj pokončati svojega nasprotnika branilca, ki ga sicer ne vidi, a ve, kje se nahaja, in če obzidje še ni prebito, bi oblegovalec lahko uporabil več strategij. Lahko bi se začel pomikati proti branitelju v upanju da ga uvidi, zamenjal meč za lok in nazadnje začel streljati na branitelja. Druga možnost bi bila, da meč obdrži in se raje premakne proti vhodu obzidja, ki ga najprej podre in s tem sprostí pot zase in ostale oblegovalce, nato pa se pomakne proti izbranemu branitelju in ga pokonča z mečem. Algoritem bi med možnostma izbral tisto, ki je zanj v danem trenutku najbolj ugodna.

GOAP hrani vnaprej pripravljen nabor akcij, ki jih akter lahko izvede. Za vsako hrani pogoje za izvedbo in njene pričakovane rezultate [7]. Za akcijo »premakni se proti nasprotniku« si denimo želimo, da se izvede le, ko ima akter opremljen meč, nasprotnik

pa je viden in ne napada. Rezultat akcije bi bil, da se akter nasprotniku dovolj približa, da lahko uporabi meč. Če želi akter izvesti akcijo »uporabi meč«, mora biti v neposredni bližini nasprotnika in obenem opremljen z mečem. Rezultat akcije je, da je nasprotnik mrtev.

Algoritem GOAP torej poišče akcije, ki se lahko izvedejo v danem stanju in sproti preverja, na kakšen način naj bi izbrana akcija spremenila stanje. Nato iz dobljenega pričakovanega stanja spet poišče akcije z ustreznimi pogoji za izvedbo in postopek ponavlja, dokler cilj ni dosežen. Navadno lahko cilj doseže na več različnih načinov. Zato ima vsaka akcija svojo funkcijo uporabnosti, ki pove, kako uporabna je izvedba akcije iz danega stanja. Algoritem najprej poišče zaporedje, katerega seštevek funkcij uporabnosti je zanj najbolj ugoden (z uporabo algoritmov za iskanje najkrajše poti v grafu, kot sta Dijkstra in A^*), nato akter izvede prvo akcijo iz zaporedja. Ker se stanje sveta sproti spreminja in ker ni nujno, da agentove akcije dosežejo pričakovan rezultat, mora agent algoritem izvajati neprestano s posodobljenimi podatki. Ciljev je lahko več, v tem primeru je smiselno, da ciljem določimo različne prioritete. Tako bo algoritem poiskal zaporedje akcij, ki je upoštevajoč prioriteto cilja najbolj ugodno [10].

3 RAZVOJ RAČUNALNIŠKE IGRE

V tem poglavju je podrobno opisan razvoj računalniške igre. Na začetku so predstavljena orodja, ki smo jih uporabljali, pri čemer je poseben poudarek na pogonu Unreal Engine. Opisana je izdelava 3D gradnikov, proceduralno grajenega terena in inteligentnih akterjev. Posebna pozornost je namenjena implementaciji algoritma GOAP v programskem jeziku C++ in izdelavi logike akterjev z uporabo diagramov Blueprint.

3.1 Uporabljena orodja

Med razvojem smo uporabljali predvsem orodje Unreal Engine [11], v katerem smo urejali 3D gradnike, jih postavljali na sceno, dodajali animacije in definirali velik del obnašanja inteligentnih akterjev. Za izdelavo 3D modelov in animacij smo poleg Unreala uporabljali tudi Blender [12], pri uvozu animacij iz knjižnice Adobe Mixamo [13] pa smo si pomagali s programom Mixamo Converter [14]. Teren smo zgradili v orodju Gaea [15], programsko kodo C++ pa pisali v razvojnem okolju Visual Studio 2022 [16].

3.1.1 Unreal Engine 5.3

Unreal Engine je igralni pogon, namenjen predvsem za izdelavo iger in ostalih interaktivnih izkušenj za najrazličnejše ciljne platforme, od osebnih računalnikov, konzol, navidezne in obogatene resničnosti, do mobilnih naprav. Zaradi visokega nivoja grafičnega realizma in hitrosti upodabljanja je primeren tudi za ustvarjanje video vsebin, grafično oblikovanje in virtualno produkcijo filmov. Razvoj logike za interaktivne izdelke poteka v grafičnem urejevalniku Blueprints, zahtevnejši uporabniki pa lahko dodatno funkcionalnost razvijajo v programskem jeziku C++, kjer imajo na voljo Unrealovo knjižnico in dostop do celotne izvirne kode pogona Unreal Engine [17]. Uporaba je za razvijalce iger popolnoma zastonj, dokler bruto prihodek od izdelka, ki vsebuje Unrealovo kodo, ne preseže določene meje [18].

S prihodom Unreal Engine 5 so razvijalci dobili ogromno izboljšav. Predvsem so razvoj iger olajšale nove tehnologije, kot na primer Nanite, ki pametno prilagaja število trikotnikov na 3D objektih glede na oddaljenost od kamere (torej imamo lahko na sceni sočasno veliko predmetov z zelo visokim številom trikotnikov, saj se bo s prilagajanjem nivoja podrobnosti namesto nas ukvarjal Unreal), sledenje žarkom (angl. *Ray-Tracing*), ki omogoča visoko mero realizma in so ga v preteklosti zaradi visoke računske zahtevnosti večinoma uporabljali le v filmih, proceduralno ustvarjanje vsebine (angl. *Procedural Content Generation*), s katerim lahko z uporabo grafičnega urejevalnika določimo pravila, po katerih bo nato računalnik sam postavil gradnike na sceno, orodje *Quixel Bridge*, prek katerega lahko dostopamo do obsežne Epicove knjižnice 3D gradnikov *Quixel Megascans*, ki je tudi del licence za Unreal Engine [17], prav tako pa nam je na voljo *Metahuman Creator*, ki nam omogoča izjemno hitro in enostavno izdelavo likov vrhunske kvalitete [19] in seveda še veliko več.

Unrealov sistem za grafično programiranje Blueprint je popoln sistem za kodiranje poteka igre, ki temelji na povezovanju vozlišč. Sistem je zelo močan in prilagodljiv, ker uporabniku ponuja praktično vse koncepte in orodja, ki jih navadno uporabljajo programerji. S specifičnimi makroji lahko programerji v programskem jeziku C++ razširijo Blueprintove funkcionalnosti [20].

Obstaja več tipov Blueprint. Med drugim poznamo razred Blueprint (angl. *Blueprint Class*), ki ga z grafičnim vmesnikom ustvarimo v *Unreal Editor*. Instance razreda lahko potem dodamo na nivo igre. Poleg osnovnega razreda Blueprint poznamo tudi Blueprint, ki vsebuje samo podatke (angl. *Data-Only Blueprint*), Blueprint nivoja (angl. *Level Blueprint*), vmesnik Blueprint (angl. *Blueprint Interface*), knjižnico Blueprint makrojev (angl. *Blueprint Macro Library*) in pripomočke Blueprint (angl. *Blueprint Utilities*). Pri urejanju razreda Blueprint se srečujemo z oknom s komponentami (angl. *Components Window*), kjer najdemo komponente, ki razred sestavljajo. V oknu lahko na primer kot komponento dodamo ostale razrede Blueprint in tudi objekte, mrežo trkov (angl. *Collision Mesh*) in gibanje (angl. *Movement Component*). Programska koda

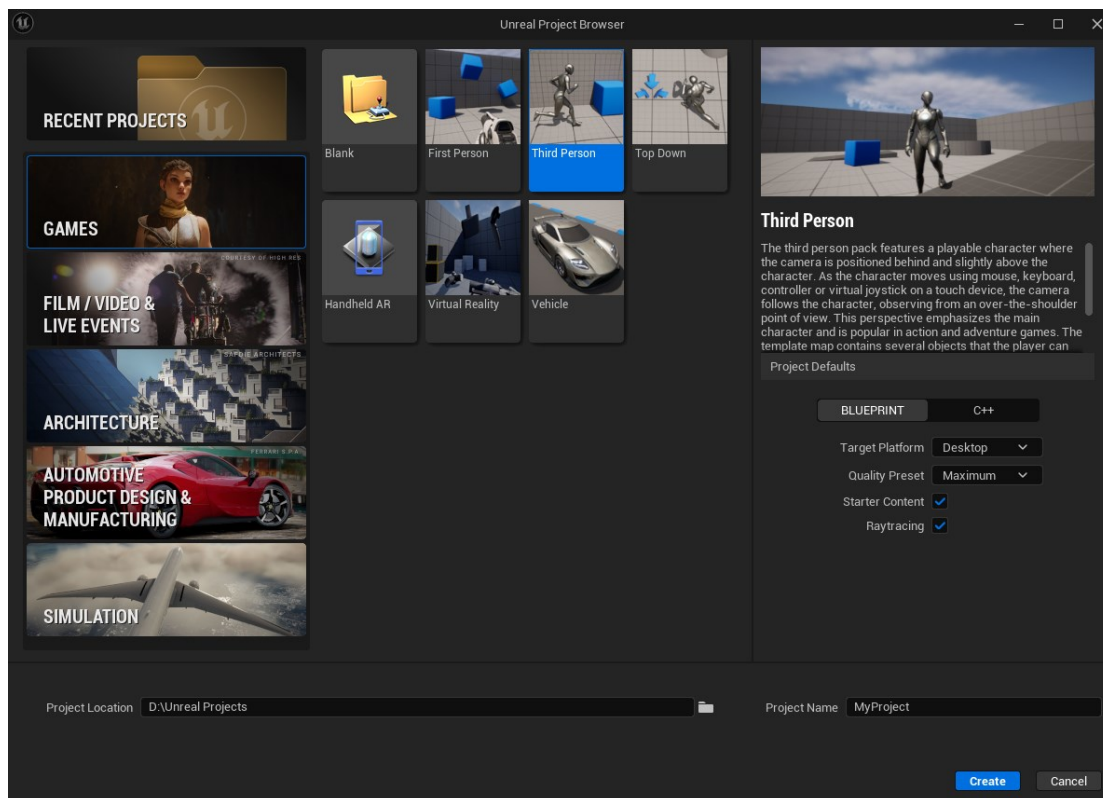
za gradnjo (angl. *Construction Script*) se izvede, ko se ustvari instanca razreda *Blueprint*. V njej se nahaja diagram poteka, v katerem definiramo inicializacijo razreda. Diagram dogodkov (angl. *Event Graph*) z uporabo dogodkov (angl. *Events*) in klicev metod (angl. *Function Calls*) izvede akcije kot odziv na dogodke, ki se prožijo tekom igre. Z njimi definiramo vedenje vseh instanc razreda *Blueprint*. Metode (angl. *Functions*) so diagrami poteka, ki pripadajo specifičnemu razredu *Blueprint* in jih lahko kličemo iz drugih diagramov razreda. Izvedba metode se začne v specifičnem vozlišču, ki ga prepoznamo po imenu metode, ki ima samo en izhod (angl. *Output Pin*), nanj vezano vozlišče se proži ob klicu metode. V spremenljivkah (angl. *Variables*) hranimo vrednosti različnih tipov, tudi reference na objekte v svetu. Njihov doseg je lahko zaseben ali javen [20].

3.1.2 Blender 3.6

Blender je brezplačen odprtokoden program za 3D ustvarjalce, ki ponuja orodja za celoten ustvarjalni proces pri računalniški grafiki. V njem lahko modeliramo, ustvarjamo ogrodja za like, animiramo, izvajamo simulacije, upodabljamo ustvarjeno sceno, ustvarjamo kompozicijo iz 2D gradnikov ali pa sledimo gibanju objektov in kamere na videoposnetkih. Če nam funkcionalnosti, ki jih ponuja Blender niso dovolj, ga lahko razširimo z lastnimi funkcionalnostmi, ki jih razvijemo v programskem jeziku Python z uporabo Blenderjeve knjižnice [21].

3.2 Vzpostavitev projekta v Unreal Engine

Za razvoj naše računalniške igre smo uporabili različico Unreal Engine 5.3.1. Ob zagonu se odpre pojavno okno *Unreal Project Browser* (slika 3.1). Ko uporabnik ustvarja nov projekt, izbere zavihek, ki je najbližje vrsti projekta. Na ta način projekt ustvari skupaj z uporabno podlago. Ker želimo izdelati tretjeosebno igro, smo v zavihku *Games* izbrali predlogo *Third Person*, v razdelku *Project Defaults* pa *Blueprint* (razrede v programskem jeziku C++ smo v igro dodali pozneje), naša ciljna platforma so osebni računalniki (*Desktop*), dodali smo začetno vsebino (*Starter Content*), ker naša grafična kartica podpira sledenje žarkom, pa smo uporabili tudi *Raytracing*.



Slika 3.1: Ustvarjanje projekta

Ob pritisku gumba *Create* se odpre Unreal Editor. Ustvarili smo nov nivo z imenom *level1* s klikom na *File*→*New Level*→*Open World*→*Create*. Unreal nam pri tem pripravi nivo, kjer že imamo površje, sonce in volumetrične učinke (slika 3.2). Ker smo ob ustvarjanju projekta izbrali predlogo *Third Person*, se ob kliku na gumb *Play* na svet postavi Unrealov privzet lik, ki ga igralec lahko premika po terenu.

V spodnjem delu uporabniškega vmesnika (slika 3.2) najdemo *Content Browser*, v okviru katerega vidimo drevesno strukturo našega projekta in z uporabo raznih filtrov in iskalnega polja iščemo gradnike, ki jih imamo shranjene v našem projektu. Če želimo uporabiti gradnik, ga z miško preprosto zapeljemo na zeleno mesto na sceni. Kadar ustvarjamo nov gradnik, v polju kjer se gradniki prikazujejo, z desnim klikom odpremo meni in izberemo želeno možnost ali pa kliknemo na gumb *Add*.



Slika 3.2: Urejevalnik nivoja

Na desnem robu urejevalnika imamo pregledovalnik (angl. *Outliner*), kjer imamo pregled nad vsemi elementi, ki smo jih postavili na nivo (slika 3.2). Elemente lahko v pregledovalniku organiziramo po mapah, kar nam lahko na kompleksni sceni močno olajša delo. Ob kliku na nek element se pod pregledovalnikom prikažejo njegove komponente, pod njimi pa lastnosti izbrane komponente, ki jih lahko urejamo.

3.3 Ustvarjanje lastnih 3D modelov

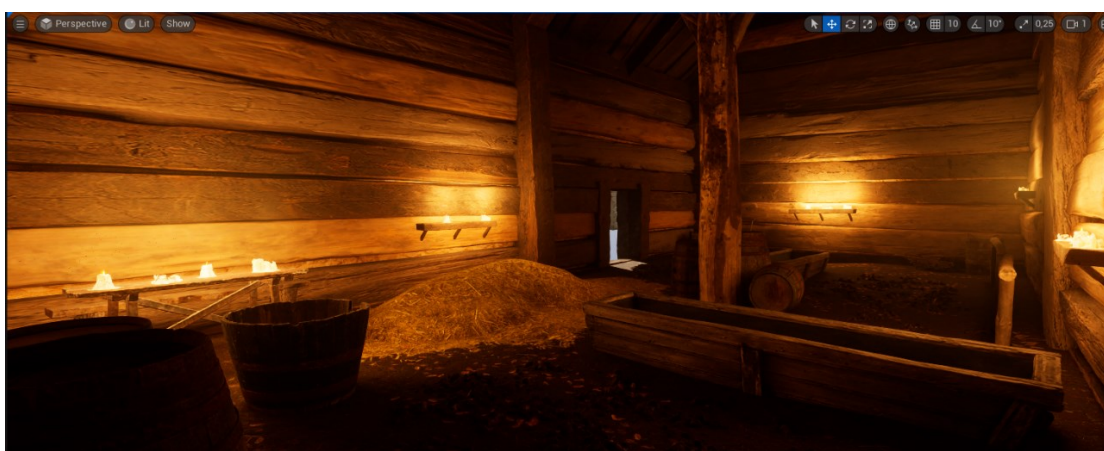
Ko imamo pripravljene vse gradnike, ki jih želimo imeti na sceni, jih lahko začnemo zlagati skupaj. Preprosto z miško prestavljamo izbran gradnik iz polja *Content Browser* na sceno in označimo objekt s klikom nanj, nato pa ga premikamo, obračamo in raztegujemo. Če želimo ustvariti kakšno simulacijo (na primer ogenj, vodo ali dim) uporabimo Unrealov *Niagara System* [22], za osvetljavo pa imamo na voljo različne tipe luči. S sestavljanjem brezplačnih gradnikov smo ustvarili vse zgradbe, ki jih bomo potrebovali kasneje pri gradnji proceduralno zgrajenega terena (slike 3.3, 3.4 in 3.5).



Slika 3.3: Izdelana miza v notranjosti polzemljanke

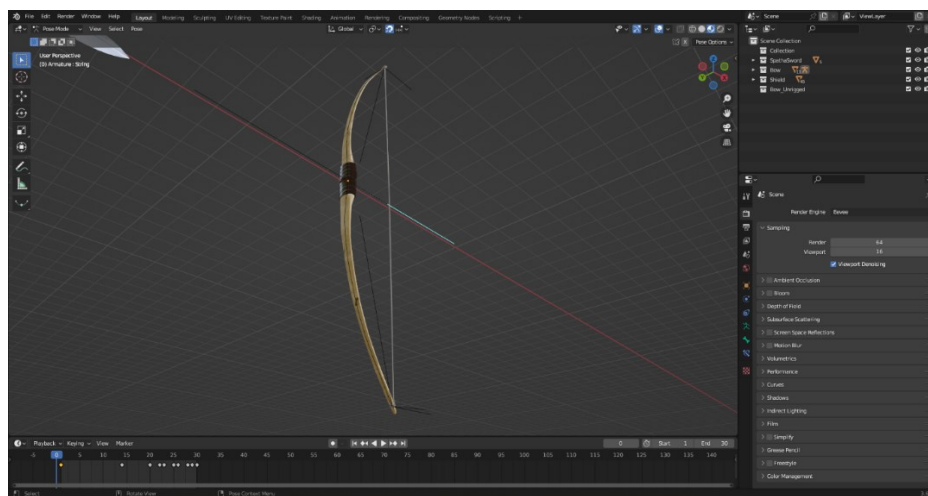


Slika 3.4: Izdelana notranjost polzemljanke

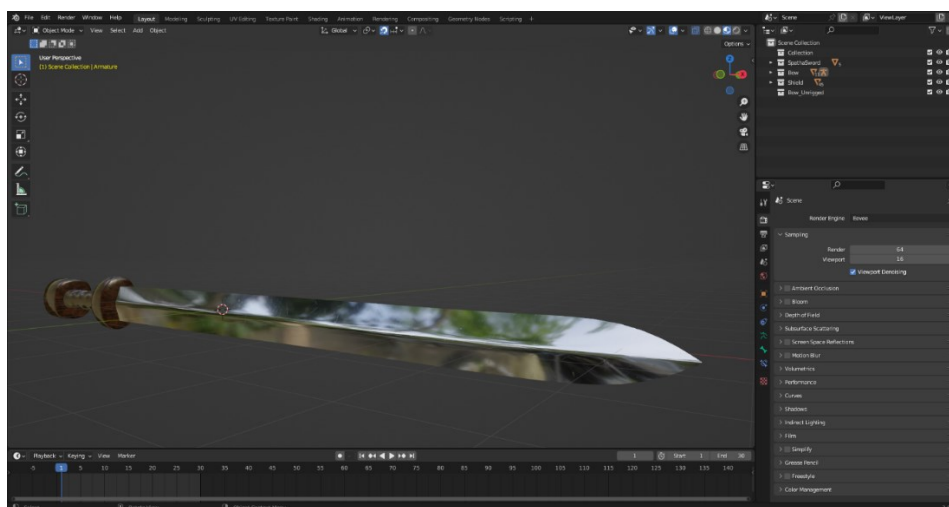


Slika 3.5: Oblikovana notranjost hleva

Za igro smo potrebovali tudi orožje, ki ga uporabljajo računalniško nadzorovani liki in igralec. Pri tem smo uporabili program *Blender 3.6*, ki je specializiran za 3D oblikovanje in animacijo in v ta namen ponuja veliko boljša orodja, kot Unreal. S premikanjem, obračanjem, raztezanjem, dodajanjem in brisanjem oglišč, robov in ploskev ter z uporabo modifikatorjev (angl. *Modifiers*) lahko preprosta telesa, kot so kocka, valj, ploskev in krogla spremenimo v karkoli si zamislimo. Dobljene modele potem poljubno pobarvamo ali pa jim dodamo ogrodje in jih nato animiramo. Izdelali smo meč (slika 3.7), lok (slika 3.6) in ščit (slika 3.8), jih pobarvali in loku dodali animacijo.



Slika 3.6: Izdelan lok v programu Blender



Slika 3.7: Izdelan meč v programu Blender



Slika 3.8: Izdelan ščit v programu Blender

Ko smo z modeli zadovoljni, jih lahko uvozimo v Unreal Engine. To lahko naredimo prek vtičnikov ali pa z izvozom v široko podprt 3D format, recimo FBX. Ko naš model uspešno izvozimo, v programu Unreal Engine v razdelku *Content Browser* kliknemo *Import* in poiščemo datoteko na našem računalniku. Če smo modelu dodali animacijo, moramo v oknu, ki se odpre, označiti *Import Animations*. Po kliku na gumb *Import* uvožen model najdemo v razdelku *Content Browser*.

3.4 Proceduralno ustvarjanje vsebine

Proceduralno ustvarjanje vsebine (angl. *Procedural Content Generation*, PCG) je tehnika, ki nam omogoča, da računalnik pripravi vsebino po določenih pravilih, pri čemer lahko za več realizma in variacij dodamo element naključja. Uporablja se, ko bi bilo ročno ustvarjanje vsebine preveč zamudno, recimo, ko želimo ustvariti zelo velik virtualen svet, ko bi radi, da je igra vsakič malce drugačna ali pa za zmanjševanje velikosti igre na disku. Prav zaradi močno omejenega spomina na takratnih računalnikih so tehniko med prvimi uporabili pri igri *Rogue* (1980) za generiranje labirintu podobnih nivojev. S PCG lahko med drugim ustvarimo nivo igre, pokrajino in animacije ali pa variacije pri kopiranih objektih [23].

3.4.1 Osnove PCG v programu Unreal Engine

Unreal Engine ima lasten urejevalnik PCG, ki je zaenkrat na voljo kot vtičnik [24]. V razdelku *Content Browser* ustvarimo nov graf PCG (*Add→PCG→PCG Graph*), ime nastavimo po želji. Ob dvojnem kliku na novo ustvarjen gradnik PCG se odpre urejevalnik vozlišč (angl. *Node Editor*), kjer urejamo graf PCG. Da bi lahko spremembe videli na našem nivoju, moramo gradnik PCG najprej postaviti na željeno mesto nivoja in mu nastaviti ustrezno velikost.

Vozlišča v urejevalniku PCG ustvarjamo z desnim klikom, nato pa izberemo željeno vozlišče. Vozlišča imajo navadno tako vhodne kot tudi izhodne točke, ki jih med seboj povezujemo s pridržanjem levega klika. Ko z levim klikom označimo vozlišče, se na desni strani urejevalnika pokažejo njegovi parametri, ki jih lahko po želji spreminjamo. Prav tako lahko izbrano vozlišče onemogočimo (*desni klik→Enable*), pri čemer se vozlišče zasenči.

Pogosto želimo vedeti, kje na sceni se nahajajo točke po izvedbi nekega vozlišča, kar nam omogočata funkcija *Debug* (*desni klik→Debug*), ki na sceni tam, kjer se nahajajo točke, privzeto nariše kocke in jih pobarva v skladu z njihovo gostoto in funkcija *Inspect* (desno od gumba *Cancel Execution* v spustnem meniju izberemo željen gradnik PCG na sceni, potem na vozlišču izberemo *desni klik→Inspect*). V spodnjem delu urejevalnika bodo v razdelku *Attributes* skupaj s svojimi lastnostmi prikazane vse točke, ki so prisotne v izbranem vozlišču. Vozlišče z vključeno funkcijo *Debug* prepoznamo po modri, *Inspect* pa po rumeni piki. Ker lahko ustvarjamo podgrafe, ki jih nato uporabljamo v ostalih grafih, imamo na voljo vozlišči *Input* in *Output*, v *Input* prejmemo podatke, v *Output* pa jih vračamo.

3.4.2 Uporaba PCG v naši igri

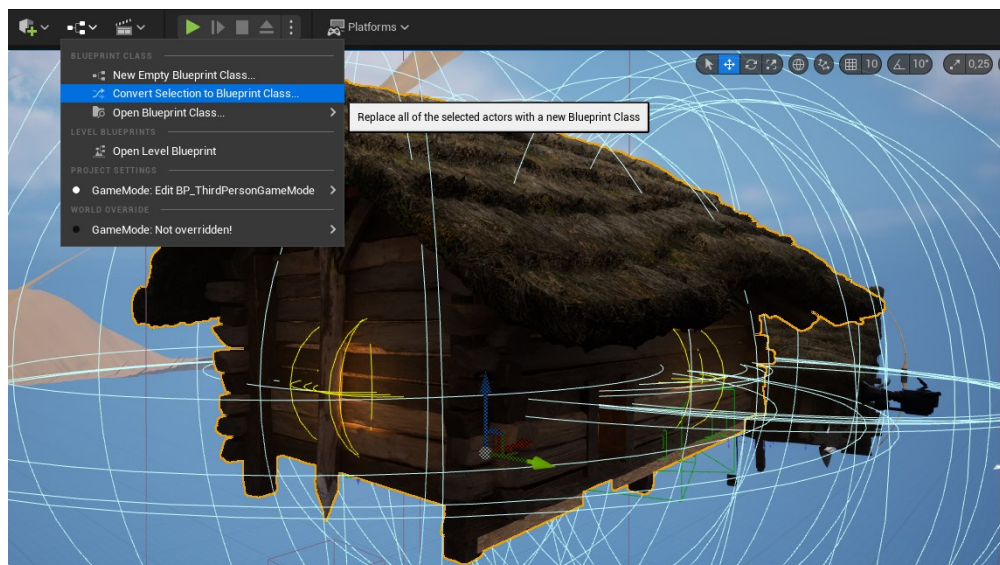
Z grafom PCG smo pusto pokrajino pokrili z vsem rastlinjem in zgradbami, razen obzidja, ki smo ga ustvarili ročno. Skozi naselbino poteka pot, ob kateri dreves, zgradb,

skal in štorov ni. Enako velja v bližini obzidja. Gostota rastja in skal je odvisna od nagiba terena (slika 3.9).



Slika 3.9: Rezultat našega grafa PCG

Ker so naše zgradbe narejene iz velikega števila različnih gradnikov, smo jih morali najprej pretvoriti v skupen razred Blueprint. Označimo vse objekte, ki model sestavljajo, nato pa v zgornjem delu urejevalnika izberemo *Convert Selection to Blueprint Class*, kot je prikazano v posnetku zaslona (slika 3.10). V pojavnem oknu izberemo *Harvest Components*, določimo *Actor* kot starševski razred, vnesemo ime in pot ter kliknemo *Select*. To naredimo za vse naše modele.



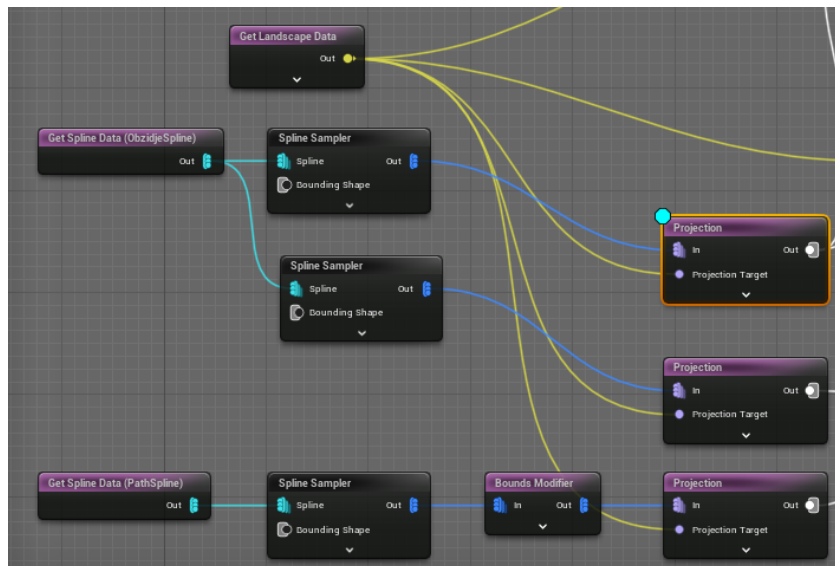
Slika 3.10: Pretvarjanje modela v razred Blueprint

Ker želimo, da bi se zgradbe postavljale le znotraj obzidja in bi radi, da skozi naselje poteka pot, smo ustvarili dve krivulji (slika 3.11). Ustvarili smo razreda Blueprint *BP_BasicSpline* in *BP_PathSpline* in vanju dodali komponento *Spline*. Na sceni smo z zaokroženo krivuljo *BP_BasicSpline* omejili obzidje, s krivuljo *BP_PathSpline* pa določili potek poti. Na sceno postavljenima krivuljama smo dodali oznaki (*Tag*) *ObzidjeSpline* in *PathSpline*, s katerima smo krivulji referencirali v grafu PCG.



Slika 3.11: Vpliv krivulj na generiranje terena

Tako smo pripravili vse potrebno za ustvarjanje grafa PCG. V nadaljevanju je opisano, kako deluje proceduralno postavljanje zgradb. Najprej smo točke, določene s krivuljama, dobili z vozliščem *Get Spline Data*, kjer smo nastavili *Actor Filter* na *All World Actors* in *Actor Selection* na *By Tag*, nato smo v polje *Actor Selection Tag* vpisali oznako ustvarjenih krivulj (*ObzidjeSpline* oziroma *PathSpline*). S *Spline Sampler* smo dobili točke, ki se v skladu z nastavljenimi parametri za vozlišče (*Dimension* je lahko nastavljen na *On Interior* ali na *On Spline*) nahajajo znotraj oziroma na krivulji. Z vozliščem *Projection* smo dobljene točke projecirali na želen cilj, pri nas je to *Landscape Data*. Točke so v vozlišču *Projection* na slikah (sliki 3.12 in 3.13) omejene s krivuljo *BP_BasicSpline*, vse pa so postavljene na površje.

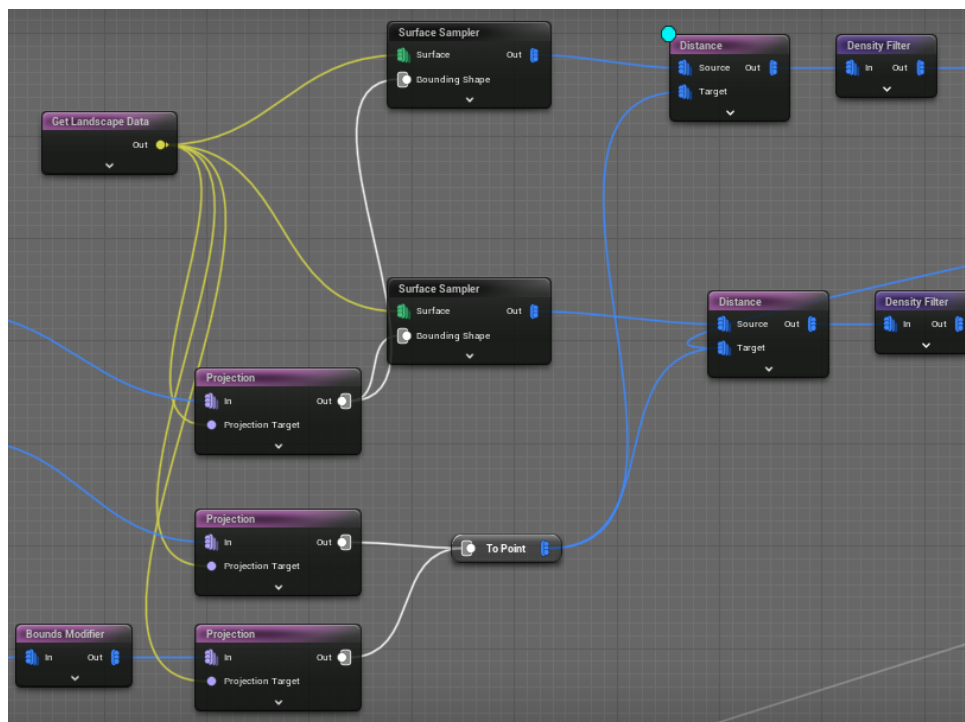


Slika 3.12: Uporaba vozlišča Projection



Slika 3.13: Vozlišče Projection prestavi vse točke iz notranjosti krivulje na tla

Z vozliščem *Distance* lahko točkam, ki jih vozlišče dobi prek vhoda *Source*, priredimo gostoto na podlagi njihove razdalje od točk z vhoda *Target*. V našem primeru smo s tem dosegli, da imajo točke ob poti in obzidju nižjo gostoto kot ostale točke na površju (sliki 3.14 in 3.15). Točke smo potem na podlagi njihove gostote filtrirali z vozliščem *Density Filter*, želimo namreč obdržati le točke z višjo gostoto (točke, ki so na sliki bele barve).



Slika 3.14: Z vozliščem Distance točkam iz vozlišča Projection nastavimo gostoto glede na oddaljenost od krivulj obzidja in poti

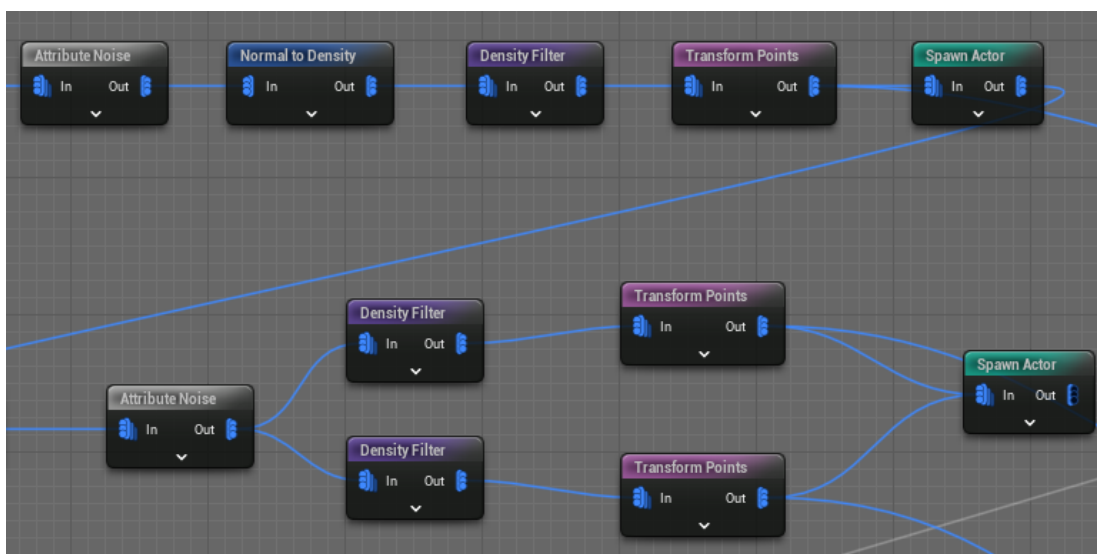


Slika 3.15: Vozlišče Distance točkam v bližini krivulj priredi nižjo, bolj oddaljenim pa višjo gostoto

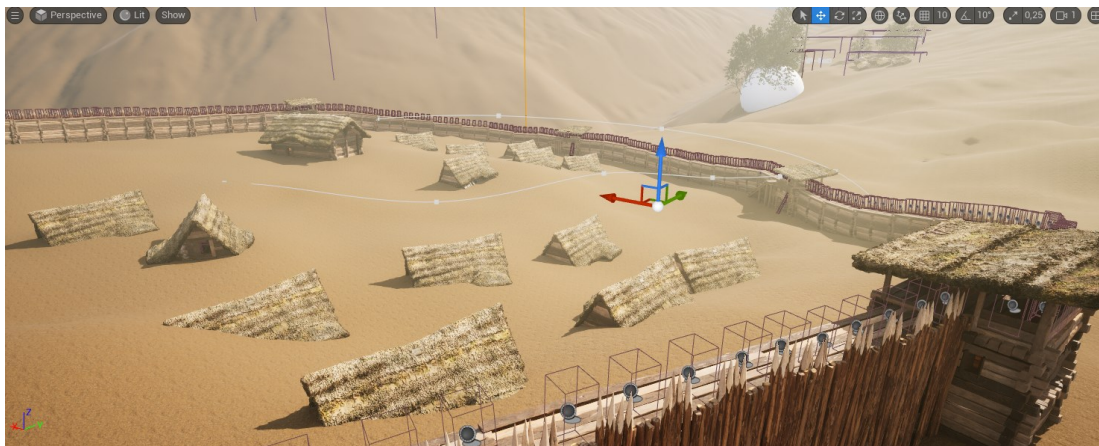
Z vozliščem *Attribute Noise* točkam naključno priredimo gostoto, z *Normal to Density* pa nastavimo gostoto točk na podlagi naklona točke (dobimo ga tako, da poiščemo vozlišče *Execute Blueprint*, potem pa nastavimo njegov *Blueprint Element Type* na

Normal to Density. Če s funkcijami, ki nam jih ponuja vtičnik PCG nismo zadovoljni, bi tu lahko uporabili lastne funkcije Blueprint). Preden na mestih, kjer se točke nahajajo, postavimo objekte, si navadno želimo točke povečati, zasukati oziroma premakniti, kar naredimo s *Transform Points*.

Predvsem Slovani so v zgodnjem srednjem veku prebivali v polzemljankah, preprostih pravokotnih kočah brez dimnika s površino od 12 do 20m², ki so jih izolirali z debelo plastjo ilovice in vkopali približno meter pod zemljo [25]. Zato smo točke, kjer smo kasneje postavili polzemljanke, pomaknili nekoliko navzdol po navpični osi, nagiba površja pa pri njih nismo upoštevali do tolikšne mere, kot pri postavljanju hlevov, ki niso vkopani. Da bi bila večina polzemljank obrnjenih z vhodom proti jugu, smo zanje uporabili dvojne vozlišč *Transform Points* (slika 3.16), enega za glavnino polzemljank in enega za majhno število polzemljank, ki so obrnjene drugače. Nato smo uporabili vozlišče *Spawn Actor*, kjer smo v nastavitvah pri *Template Actor Class* izberali razred Blueprint našega modela (sliki 3.16 in 3.17). Na ta način smo izdelali algoritem, s katerim lahko brez težav zgradimo zelo velik svet oziroma s spreminjanjem parametrov posameznih vozlišč in oblike krivulj na istem delu sveta postavimo nepredstavljivo število različnih naselbin.



Slika 3.16: Uporaba vozlišč *Spawn Actor* in *Transform Points*



Slika 3.17: Z vozliščema Spawn Actor postavljene zgradbe

3.5 Inteligentni akterji v okviru principa delovanja igre

Želimo ustvariti tretjeosebno računalniško igro, postavljeno v utrjeno srednjeveško naselbino. Igralec igro prične znotraj obzidja kot branitelj, pri čemer mu pomagajo ostali inteligentni akterji. Na drugi strani se nahajajo oblegovalci, katerih cilj je prebiti eno izmed obeh vrat obzidja. To storijo tako, da po vratih udarjajo z mečem (slika 3.18).



Slika 3.18: Prizor iz igre, ko se akterji združujejo pred vrati obzidja

Pri tem jih branitelji z zgornjega dela obzidja poskušajo zadeti z lokom, na kar oblegovalci odgovorijo. Na sliki (slika 3.19) je oblegovalec streljal na branitelja, zato ga je branitelj pokončal. Ko vrata obzidja padejo, se tabora udarita med seboj (slika 3.20). Pri tem lahko igralec in računalniško nadzorovani liki spreminjajo orožje.



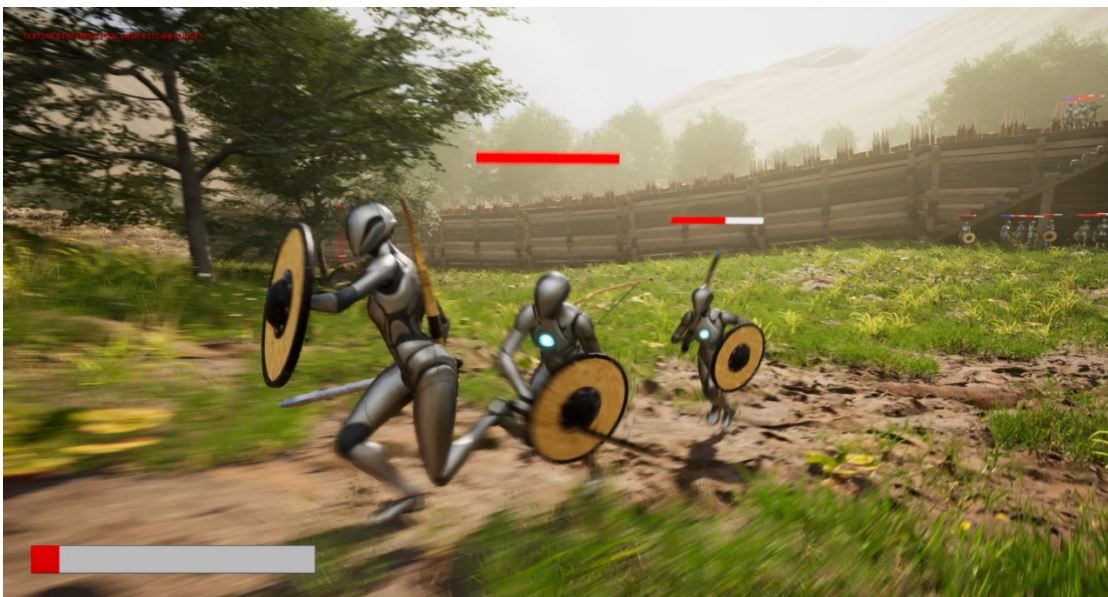
Slika 3.19: Branitelj (na levi strani, z lokom v roki) pokonča oblegovalca (desno)



Slika 3.20: Vrata obzidja padejo, akter (desno) zasleduje igralca (levo)

Igralec ima lahko opremljen meč in ščit oziroma lok (vsako orožje lahko uporablja neomejeno). Če v trenutku, ko vanj prileti puščica oziroma ga nasprotnik udari z mečem uporabi ščit, se njegovo zdravje ne zmanjša. Sicer se zmanjša za 10% ob udarcu z mečem oziroma za 20%, ko vanj prileti sovražna puščica. Ker zdravja ni mogoče napolniti je cilj igre preživeti čim dlje časa. Prijateljskega ognja ni.

Na enak način se vojskujejo liki, ki jih nadzoruje računalnik. Da igra ne bi bila preveč težavna, večinoma uporabljajo meč, čeprav bi z lokom veliko lažje pokončili svoje nasprotnike (slika 3.21). Število akterjev, ki hkrati napadajo istega nasprotnika, je omejeno. Ko računalniško nadzorovanemu liku ali igralcu zmanjka življenja, je igre zanj konec (slika 3.22).



Slika 3.21: Prizor iz igre, ko akter (v sredini) udari igralca (na levi)



Slika 3.22: Konec igre, ker je igralcu zmanjkalo življenja

3.5.1 Implementacija algoritma GOAP

Čeprav je Unrealov grafični programski jezik Blueprints splošno uporaben, je zaradi preglednosti in hitrosti razvoja za naprednejše algoritme bolje uporabiti programski jezik C++. V ta namen smo uporabljali integrirano razvojno okolje *Visual Studio 2022*, ki smo ga konfigurirali za delo z našo različico Unreal Engine.

V Unrealu smo ustvarili nov razred programskega jezika C++ (*Tools*→*New C++ Class...*) z imenom *CPP_GoapPlanner*, ki deduje iz razreda *Character*. Unreal nam za novo ustvarjen razred pripravi datoteki CPP in H, v katerih najdemo konstruktor razreda in metode *BeginPlay*, *Tick* in *SetupPlayerInputComponent* (slika 3.23).

```

// Sets default values
ACPP_GoapPlanner::ACPP_GoapPlanner()
{
    // Set this character to call Tick() every frame. You can turn this off to improve performance if you don't need it.
    PrimaryActorTick.bCanEverTick = true;
}

// Called when the game starts or when spawned
void ACPP_GoapPlanner::BeginPlay()
{
    Super::BeginPlay();
}

// Called every frame
void ACPP_GoapPlanner::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);
}

// Called to bind functionality to input
void ACPP_GoapPlanner::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

```

Slika 3.23: Privzeto ustvarjen Unrealov razred v programskem jeziku C++

V tem razredu želimo izdelati metode in spremenljivke, ki smo jih kasneje uporabili v razredu Blueprint naših akterjev, ki deduje iz *CPP_GoapPlanner*. Deklarirali smo jih v zaglavni (angl. *header*) datoteki razreda C++, njihov doseg je javen (slika 3.24). Primer takšne metode je *GOAPGetKillPlayerCost*.

```

UFUNCTION(BlueprintCallable, Category = "Goal Oriented Action Planning") void GOAPExecuteDijkstra
(bool playerInSight, bool playerInSwordRange, bool playerInBowRange,
6 Blueprint references
    bool playerUsingSword, bool playerUsingShield, bool swordEquipped, bool bowEquipped, bool onTheHighGround, bool playerDead,
    bool doorOpened, bool inDoorRange);

UFUNCTION(BlueprintCallable, Category = "Goal Oriented Action Planning") int GOAPGetKillPlayerCost(int goalCostMultiplier);

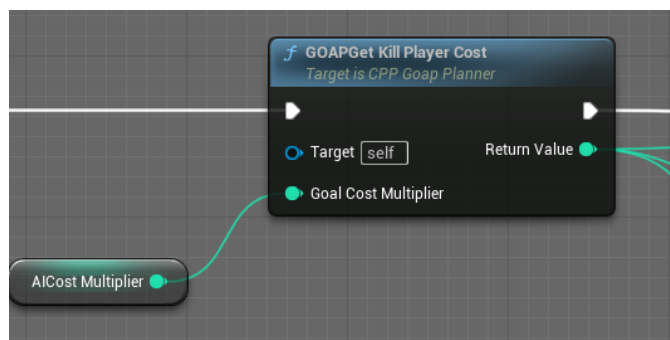
UFUNCTION(BlueprintCallable, Category = "Goal Oriented Action Planning") void GOAPTriggerNextActionBlueprintEvent();

```

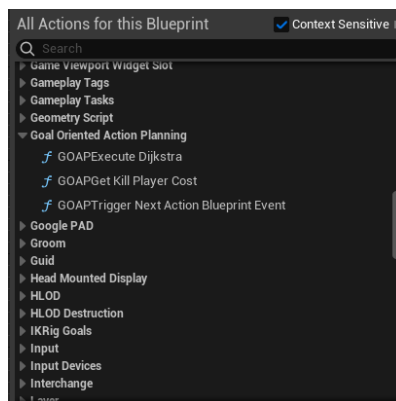
Slika 3.24: Metode C++, dosegljive iz razreda Blueprint

V razredu Blueprint smo ob klicu poslali parameter *goalCostMultiplier* tipa *int*, s katerim bo metoda pomnožila skupno ceno načrtovanih akcij, ki jih bo agent izvedel, nato pa jo vrnila kot izhodni parameter tipa *int* (slika 3.25). Ker želimo, da je metoda na voljo v urejevalniku Blueprint, uporabimo makro *UFUNCTION* in navedemo *BlueprintCallable* kot specifikator funkcije (slika 3.24). Funkcijo smo tako lahko klicali iz razreda Blueprint, implementirali pa smo jo v C++. Ob desnem kliku v urejevalniku Blueprint se uporabniku funkcije in spremenljivke, ki jih ima na voljo, izpišejo po kategorijah (slika 3.26), zato pri deklaracijah zaradi preglednosti navedemo kategorijo

Category = »Goal Oriented Action Planning« (slika 3.24). Tu definirane metode v razredu Blueprint uporabljamo tako, da najprej pokličemo metodo *GOAPExecuteDijkstra*, v katero pošljemo parametre tipa *boolean*, s katerimi opišemo stanje sveta, ki ga opazuje agent. Nato kličemo prej omenjeno metodo *GOAPGetKillPlayerCost*, nazadnje pa, če smo z vrnjeno vrednostjo zadovoljni, kličemo *GOAPTriggerNextActionBlueprintEvent*. Metoda bo izvedla prvo akcijo iz pripravljenega načrta za dosego cilja tako, da bo sprožila dogodek Blueprint, kjer je akcija definirana [26].



Slika 3.25: Uporaba metode C++ v razredu Blueprint



Slika 3.26: Iskanje metod v urejevalniku Blueprint

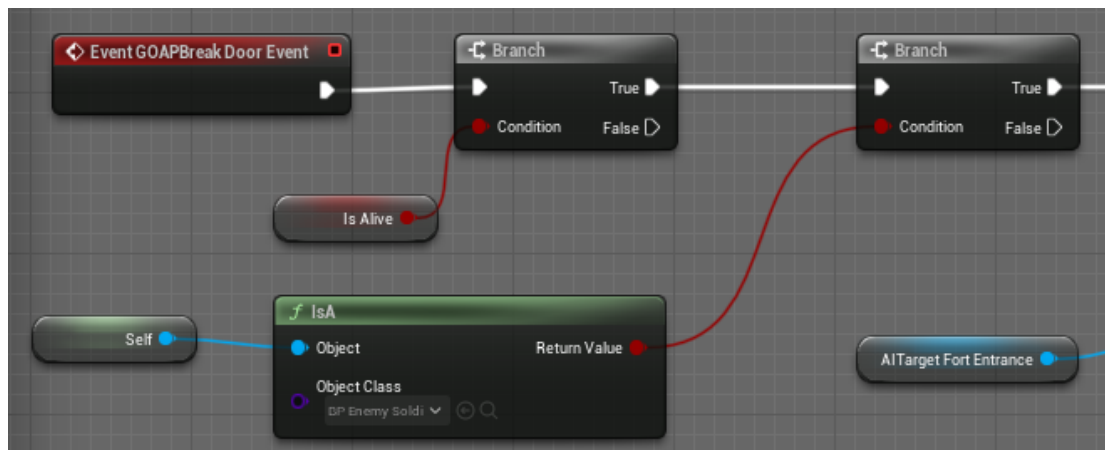
Dogodke izdelamo podobno kot funkcije, le da uporabimo specifikator *BlueprintImplementableEvent* (slika 3.27). Dogodkov nismo implementirali v C++, ampak v razredu Blueprint (slika 3.28).

```

UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPEquipSwordEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPEquipBowEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPSwordAttackEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPShieldBlockEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPBowAttackEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPMoveToPlayerSwordRangeEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPMoveToPlayerBowRangeEvent();
6 Blueprint references
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPMoveToHighGroundEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPMoveToDoorRangeEvent();
6 Blueprint references
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPBreakDoorEvent();
UFUNCTION(BlueprintImplementableEvent, Category = "Goal Oriented Action Planning") void GOAPDoNothingEvent();

```

Slika 3.27: Deklaracija dogodkov Blueprint



Slika 3.28: Implementacija dogodka GOAPBreakDoorEvent

Razredne spremenljivke, do katerih bi radi dostopali iz razreda Blueprint, smo deklarirali z makrom *UPROPERTY* (slika 3.29). Pri spremenljivki *GOAP_MAX_VALUE* smo želeli, da je uporabniku dosegljiva le za branje, zato smo uporabili specifikator *BlueprintReadOnly*. Pri ostalih spremenljivkah smo s specifikatorjem *EditAnywhere* uporabniku omogočili urejanje spremenljivke iz oken Unrealovega uporabniškega vmesnika, z *BlueprintReadWrite* pa branje in pisanje v spremenljivko iz urejevalnika Blueprint (slika 3.30) [27]. Te spremenljivke se uporabljajo pri določanju cene izvedbe posameznih akcij. Ustvarili smo tudi javni razredni spremenljivki *isAnEnemy* tipa *bool* in *thisActorsDijkstraStartNode* s kazalcem na vozlišče grafa vozlišč, ki pove, v katerem stanju se agent trenutno nahaja (slika 3.29). Spremenljivki iz Unreal Editorja nista dosegljivi, do njiju lahko dostopamo le v programski kodi C++.

```

UCLASS()
22 derived Blueprint classes
class KARANTANIJA_API ACPG_GoapPlanner : public ACharacter
{
    GENERATED_BODY()

public:
    // Sets default values for this character's properties
    ACPG_GoapPlanner();

    bool isAnEnemy;
    WorldStateGraphNode* thisActorsDijkstraStartNode;

    UPROPERTY(BlueprintReadOnly, Category = "Goal Oriented Action Planning") int GOAP_MAX_INT_VALUE = 2147483647;
    Changed in 0 Blueprints

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPThisSoldierToPlayerDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPThisSoldierToDoorDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPThisSoldierToHighGroundDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPDoorToHighGroundDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPDoorToPlayerDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPHighGroundToPlayerDistance;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPBowAttackRange;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPEquipSwordCost;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPEquipBowCost;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPSwordAttackCost;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPShieldBlockCost;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPBowAttackCost;
    Changed in 0 Blueprints
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Goal Oriented Action Planning") int GOAPBreakDoorCost;

```

Slika 3.29: Deklaracije spremenljivk, dosegljivih iz razreda Blueprint



Slika 3.30: Spreminjanje vrednosti spremenljivke GOAPBowAttackRange

3.5.1.1 Pomožni razredi

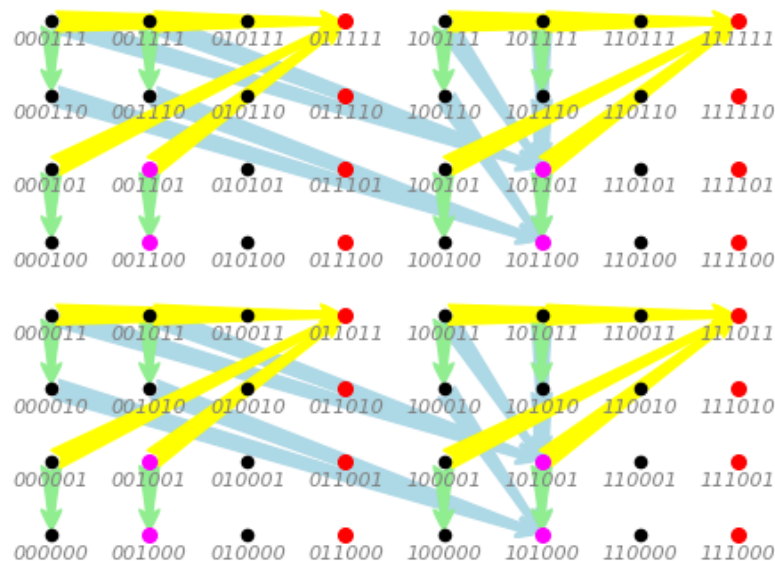
Unreal se je iz zgodovinskih razlogov izognil neposredni uporabi standardnih knjižnic C++, namesto njih nam ponuja lastne funkcionalnosti [28]. Zato smo med pisanjem programske kode nekatere funkcionalnosti razvili sami. Za dodatne razrede, ki smo jih pri implementaciji algoritma potrebovali, smo ustvarili datoteki *GoapPlannerCustomClasses.h* z deklaracijami in *GoapPlannerCustomClasses.cpp* z definicijami.

Na skici (slika 3.31) je prikazana ilustracija delovanja akcij opisane implementacije algoritma GOAP. Zaradi preglednosti je število parametrov stanja sveta enako 6, vozlišča pa povezujejo le tri akcije (rumene, zelene in modre puščice). Pri naši implementaciji je število parametrov stanja sveta in število akcij večje, pogoji za izvedbo in rezultati akcij pa so definirani drugače. Akter ima pri naši igri samo en cilj, ker pa algoritem GOAP lahko deluje pri večjem številu ciljev z različnimi prioritetami, sta na skici prikazana dva (rdeče in vijolične točke). Algoritem glede na število parametrov pripravi vsa možna vozlišča v grafu (na skici $2^6 = 64$).

Akcije so pravzaprav funkcije, katerih definicijsko območje zajema vsa števila, katerih binarna predstavitev ustreza določeni shemi (pogoji akcije). Tem številom priredi po eno število, ki ustreza določeni shemi (rezultati akcije), pri čemer nedefinirane bite zamenja z biti števila iz definicijskega območja. Na skici rumena akcija številom $*0***1$ priredi število $*11*1*$, zelena akcija številom $*0***1$ priredi $*0***0$, modra pa številom $*0**1*$ priredi $1*1*0*$. Akterjevi cilji so vozlišča, ki so prav tako definirana s shemami. Vijoličen cilj določa shema $*01*0*$, rdeč cilj pa shema $*11***$. Nekatere akcije so pri oblegovalcih določene drugače, kot pri braniteljih.

Akter v takšnem grafu najprej poišče vozlišče, ki ga določa trenutno stanje sveta. Nato iz njega zažene algoritem iskanja najkrajše poti v grafu, pri čemer se med vozlišči pomika s pomočjo akcij, ki jih povezujejo. Pri tem seštevajo cene izvedbe posameznih akcij (pri naši implementaciji so akcije predstavljene z razredom, ki vsebuje metodo, ki vrača ceno izvedbe akcije. Cena posamezne akcije se tekom igre lahko spreminja, na primer glede na akterjevo oddaljenost od nasprotnika oziroma od vhoda obzidja).

Vsako najdeno pot do cilja nato pomnoži s številom, ki določa prioriteto najdenega cilja. Na ta način se bo akter prej odločil za cilj z višjo prioriteto, čeprav je skupna cena akcij razmeroma velika. Na skici vidimo, da iz nekaterih izhodišč akter ne more doseči nobenega cilja. V takšnem primeru se pri naši igri začne sprehajati naokoli, dokler stanje sveta ne določa vozlišča znotraj povezane komponente grafa, ki vsebuje cilj.



Slika 3.31: Ilustracija delovanja akcij pri GOAP

Stanje sveta, v katerem se akter nahaja, je predstavljeno s strukturo *WorldState*, ki vsebuje 11 parametrov, s katerimi stanje opišemo (slika 3.32). Parametri so lahko resnični ali neresnični, vendar smo pri implementaciji potrebovali tudi tretjo vrednost, ko parameter ni določen. Zato smo namesto tipa *bool* pri spremenljivkah uporabili tip *int*, ki bo hranil vrednosti 0, 1 ali 2. Privzeto so vsi parametri nastavljeni na 2.

```

struct WorldState {
    //lahko je 0=false, 1=true ali 2=N/A
    int playerInSight = 2;
    int playerInSwordRange = 2;
    int playerInBowRange = 2;
    int playerUsingSword = 2;
    int playerUsingShield = 2;
    int swordEquipped = 2;
    int bowEquipped = 2;
    int onTheHighGround = 2;
    int playerDead = 2;
    int doorOpened = 2;
    int inDoorRange = 2;
};

```

Slika 3.32: Struktura WorldState

Nato smo definirali posamezne akcije, ki jih akter lahko izvede. Akcije imajo v algoritmu GOAP definirane pogoje za izvedbo, pričakovane rezultate ob izvedbi akcije in metodo, ki vrača ceno izvedbe akcije iz danega stanja sveta. Zato smo deklarirali abstrakten

razred *Action*, iz katerega dedujejo razredi, ki predstavljajo posamezne akcije (slika 3.33). Razred vsebuje tudi referenco na instanco razreda *ACPP_GoapPlanner*, spremenljivko, ki hrani njegovo ime in metodo *triggerBlueprintEvent*, s katero sprožimo dogodek Blueprint, ki definira akterjevo obnašanje ob izvedbi akcije.

```
class Action {
public:
    ACPP_GoapPlanner* currentClassInstance;
    FString name;

    WorldState preconditions;
    WorldState results;

    Action(FString name, ACPP_GoapPlanner* goapPlannerInstance);

    virtual int getPerformCost(WorldState* startpointWorldState, bool startpointEqualsDijkstraStartNode) = 0;
    virtual void triggerBlueprintEvent() = 0;

    virtual ~Action();
};

class ActionEquipSword : public Action {
public:
    ActionEquipSword(ACPP_GoapPlanner* goapPlannerInstance);

    int getPerformCost(WorldState* startpointWorldState, bool startpointEqualsDijkstraStartNode) override;
    void triggerBlueprintEvent() override;

    ~ActionEquipSword() override;
};
```

Slika 3.33: Razred Action in izpeljan razred ActionEquipSword

Pogoji za izvedbo akcij in njihovi rezultati se lahko razlikujejo, če akter pripada oblegovalcem oziroma braniteljem obzidja, s čemer dosežemo skupinsko obnašanje. Akcijo *ActionMoveToHighGround* (slika 3.34) denimo, lahko oblegovalec izvede samo, če je njegova vojska že uspela prebiti obzidje. Za vse akterje pa velja, da lahko akcijo izvedejo le, kadar se ne nahajajo na zgornjem delu obzidja in imajo v roki meč namesto loka (slika 3.34). V definiciji akcije akterju obljubimo, da bo po njeni izvedbi stal na zgornjem delu obzidja, s katerega bo lahko videl svojega nasprotnika (kar seveda ni nujno res). Pogoji za izvedbo in obljubljeni rezultati akcije so predstavljeni kot struktura *WorldState*, ki ima privzeto vse vrednosti nastavljene na 2, tiste ki bi jih radi upoštevali, pa v konstruktorju posamezne akcije postavimo na 0 oziroma 1.

Če se bo akcija izvedla iz trenutnega stanja sveta, bo njena cena (ki jo vrača metoda *getPerformCost*) razdalja akterja do visokega dela obzidja. V primeru, da se cena akcije

računa za izvedbo iz stanja sveta, ki ga lahko akter doseže po izvedbi drugih akcij, mora metoda namesto trenutnega akterjevega položaja upoštevati položaj akterja v bodočem stanju sveta. Če bo torej v bodočem stanju stal pred vrati, mora biti cena izvedbe akcije iz bodočega stanja določena kot razdalja od visokega dela obzidja do vrat (namesto od visokega dela obzidja do akterjevega trenutnega položaja). Če se bo akter v bodočem stanju nahajal pred svojim nasprotnikom, je izvedba akcije iz bodočega stanja enaka razdalji od visokega dela obzidja do nasprotnika (namesto do trenutnega položaja akterja. Ker akter nasprotnikovih namer ne pozna, privzame, da nasprotnik položaja ne bo spremenil). Cene akcij se seveda tekom igre ustrezno spreminjajo. Ob izvedbi akcije se sproži v razredu Blueprint implementiran dogodek *GOAPMoveToHighGroundEvent*. Ostale akcije so definirane na podoben način.

```

~ ActionMoveToHighGround::ActionMoveToHighGround(ACPP_GoapPlanner* goapPlannerInstance) :
    Action(FString(TEXT("Move to the High Ground")), goapPlannerInstance) {
    if (currentClassInstance->isAnEnemy) {
        this->preconditions.doorOpened = 1;
    }
    this->preconditions.onTheHighGround = 0;
    this->preconditions.swordEquipped = 1;
    this->preconditions.bowEquipped = 0;

    this->results.onTheHighGround = 1;
    this->results.playerInSight = 1;
}

~ int ActionMoveToHighGround::getPerformCost(WorldState* startpointWorldState, bool startpointEqualsDijkstraStartNode) {
    if (startpointEqualsDijkstraStartNode) {
        return this->currentClassInstance->GOAPThisSoldierToHighGroundDistance;
    }

    if (startpointWorldState->inDoorRange == 1) {
        return this->currentClassInstance->GOAPDoorToHighGroundDistance;
    }
    else if ((startpointWorldState->playerInBowRange == 1) || (startpointWorldState->playerInSwordRange == 1)) {
        return this->currentClassInstance->GOAPHighGroundToPlayerDistance;
    }
    else {
        return this->currentClassInstance->GOAPThisSoldierToHighGroundDistance;
    }
}

~ void ActionMoveToHighGround::triggerBlueprintEvent() {
    this->currentClassInstance->GOAPMoveToHighGroundEvent();
}

ActionMoveToHighGround::~ActionMoveToHighGround() {}

```

Slika 3.34: Razred ActionMoveToTheHighGround

Pri naši implementaciji z Dijkstrinim algoritmom preiskujemo graf vseh možnih stanj sveta, definiranih s strukturo *WorldState*. Vozlišče v grafu je predstavljeno z instanco razreda *WorldStateGraphNode* (slika 3.35), ki vsebuje:

- unikaten *WorldState* s katerim je vozlišče določeno,
- seznam povezav do sosednjih vozlišč (povezavo, predstavljeno s strukturo *WorldStateGraphConnection*, določata kazalec na akcijo, ki vozlišči povezuje, in indeks vozlišča v grafu, kamor povezava poteka),
- kazalec na vozlišče, iz katerega je Dijkstrin algoritem prišel do njega,
- kazalec na akcijo, prek katere je Dijkstrin algoritem prišel do vozlišča
- dolžino najkrajše poti iz trenutnega stanja sveta,
- vrednosti, ki nam povejo, če je Dijkstrin algoritem vozlišče že našel oziroma že obiskal,
- indeks vozlišča v prednostni vrsti, kamor Dijkstrin algoritem dodaja vozlišča, ki jih na novo odkrije.

V konstruktorju razreda *WorldStateGraphNode* so vse vrednosti nastavljene na 0, *nullptr* oziroma *false*.

```

struct WorldStateGraphConnection {
public:
    Action* connectingAction;
    int endpointNodeIndex;
};

class WorldStateGraphNode {
public:
    WorldState worldState;
    TArray<WorldStateGraphConnection> adjacentVerticesLinks;

    WorldStateGraphNode* dijkstraPreviousNode;
    Action* dijkstraPreviousNodeConnectingAction;
    int dijkstraShortestPathFromStart;
    bool dijkstraVisited;
    bool dijkstraDiscovered;
    int nodeIndexInBinaryHeapArray;

    WorldStateGraphNode();
};

```

Slika 3.35: Struktura *WorldStateGraphConnection* in razred *WorldStateGraphNode*

3.5.1.2 Inicializacija

Preden akter začne izvajati algoritem GOAP, mora biti graf stanj in povezav pripravljen. To se zgodi v konstruktorju razreda *CPP_GoapPlanner*, ki se kliče, ko se na sceni pojavi

nov akter, ki iz njega deduje, oziroma ob začetku igre, če ga na nivo postavimo ročno [29].

Najprej se kliče metoda *setDefaultActionsCosts*, ki nastavi vse spremenljivke, ki se uporabljajo pri določanju cen akcij na privzete vrednosti. Ker so odvisne od položaja akterja v svetu in položaja njegovih ciljev, jih tekom igre priročno spreminjamo v razredu Blueprint.

Nato se kliče metoda *fillActionsList* (slika 3.36), ki v spremenljivko *isAnEnemy* shrani logično vrednost *true*, če je akter igralcu sovražen. Če algoritem iz danega stanja ne uspe najti poti do izbranega cilja, se izvede akcija *ActionDoNothing*, kazalec do nje se tu shrani v spremenljivko *defaultActionToExecute*. Nato metoda ustvari po eno instanco vsake akcije in kazalce do njih doda v seznam *TArray<Action*> actionsList*.

```
void ACPG_GoapPlanner::fillActionsList() {
    FString currentClassName = this->GetName();
    if (currentClassName.Contains("enemy", ESearchCase::IgnoreCase)) {
        isAnEnemy = true;
    }
    else {
        isAnEnemy = false;
    }

    defaultActionToExecute = new ActionDoNothing(this);

    actionsList.Empty();

    actionsList.Add(new ActionEquipSword(this));
    actionsList.Add(new ActionEquipBow(this));
    actionsList.Add(new ActionSwordAttack(this));
    actionsList.Add(new ActionShieldBlock(this));
    actionsList.Add(new ActionBowAttack(this));
    actionsList.Add(new ActionMoveToPlayerSwordRange(this));
    actionsList.Add(new ActionMoveToPlayerBowRange(this));
    actionsList.Add(new ActionMoveToPlayerBowRangeWhenDoorClosed(this));
    actionsList.Add(new ActionMoveToHighGround(this));
    actionsList.Add(new ActionMoveToDoorRange(this));
    actionsList.Add(new ActionBreakDoor(this));
}
```

Slika 3.36: Metoda fillActionsList

Ko se *actionsList* napolni, lahko napolnimo še seznam *TArray<WorldStateGraphNode> adjacencyList*, ki predstavlja graf vseh možnih stanj sveta in akcij, ki jih mora akter izvesti, da preide iz posameznega stanja v drugega. Najprej seznam napolnimo z vsemi možnimi stanji sveta (vse možne dvojiške kombinacije parametrov strukture *WorldState*), ki jih rekurzivno poiščemo v metodi *fillAdjacencyList* (slika 3.37). Vanjo ob

prvem klicu kot parameter pošljemo nov objekt tipa *WorldState* in parameter *propertyIndex = 0*. Metoda z rekurzijo poišče vse dvojiške kombinacije števila, ki ga zapišemo z *WORLDSTATE_NUMBER_OF_PARAMETERS* biti (število parametrov v strukturi *WorldState* je 11). Ko najde novo stanje sveta, ustvari pripadajoče vozlišče (instanca razreda *WorldStateGraphNode*), ki mu nastavi spremenljivko *worldState* na *currentState* in ga doda v *adjacencyList*. Ker našo strukturo *WorldState* sestavlja 11 parametrov, ob koncu rekurzije v *adjacencyList* najdemo $2^{11} = 2048$ vozlišč, vsako predstavlja unikatno stanje sveta.

```
void ACPG_GoapPlanner::fillAdjacencyList(int propertyIndex, WorldState currentState) {
    if (propertyIndex == 0) {
        adjacencyList.Empty();
    }

    if (propertyIndex == WORLDSTATE_NUMBER_OF_PARAMETERS) {
        WorldStateGraphNode newGraphNode = WorldStateGraphNode();
        newGraphNode.worldState = currentState;

        adjacencyList.Add(newGraphNode);
    }
    else {
        setWorldStateParameterValueByIndex(&currentState, propertyIndex, 0);
        fillAdjacencyList(propertyIndex + 1, currentState);

        setWorldStateParameterValueByIndex(&currentState, propertyIndex, 1);
        fillAdjacencyList(propertyIndex + 1, currentState);
    }
}
```

Slika 3.37: Metoda fillAdjacencyList

Ko imamo graf napolnjen z vozlišči, moramo poiskati le še povezave med njimi. Povezave med vozlišči so akcije, ki se ob stanju sveta, opisanem v nekem vozlišču lahko izvedejo, povezujejo pa ga z vozlišči, opisanimi s stanji, ki nastanejo kot rezultat izvedbe akcije. Za vsako akcijo v seznamu *actionsList* kličemo metodo *FindWorldStateStartpointsForActions* (slika 3.38), kamor ob klicu pošljemo kazalec na akcijo, *propertyIndex* nastavimo na 0, kot vhodni parameter *WorldState* *currentWorldState* pa ob klicu pošljemo *WorldState* s privzetimi vrednostmi parametrov. Metoda podobno kot metoda *fillAdjacencyList* rekurzivno poišče vsa stanja sveta, iz katerih se dana akcija lahko izvede. Pri tem spreminja le parametre, ki v pogojih akcije (*WorldState preconditions*) niso definirani (njihova vrednost je 2), ostale parametre prepíše iz spremenljivke *preconditions* dane akcije. Ker so vozlišča v *adjacencyList* urejena po vrsti glede na njihov *WorldState*, lahko zmeraj, ko metoda

najde nov možen *WorldState*, s pretvorbo njegovega dvojiškega zapisa (ki ga dobimo, če po vrsti zapišemo vrednosti parametrov v *currentState*) v desetiški zapis dobi indeks vozlišča z najdenim *WorldState* v seznamu *adjacencyList*. Ko metoda *findEndpointNodeIndex* popravi parametre *WorldState* najdenega vozlišča (parametre, ki se z izvedbo akcije spremenijo, postavi na vrednosti iz spremenljivke *results* pri dani akciji), dobi vozlišče, do katerega pridemo iz najdenega vozlišča z izvedbo akcije. Povezavo (sestavljeno iz kazalca na akcijo in indeksa sosednjega vozlišča) doda v seznam *adjacentVerticesLinks*, ki ga hrani najdeno vozlišče.

```
void ACPG_GoapPlanner::FindWorldStateStartpointsForActions(Action* actionFromActionList, int propertyIndex, WorldState currentState) {
    int currentParameterValue = getWorldStateParameterValueByIndex(&actionFromActionList->preconditions, propertyIndex);

    if (propertyIndex == WORLDSTATE_NUMBER_OF_PARAMETERS) {
        //pretvori iz binary v numeričen zapis indeksa
        int numericNodeIndex = 0;
        for (int i = 0; i < WORLDSTATE_NUMBER_OF_PARAMETERS; i++) {
            numericNodeIndex += (calculatePower(2, (WORLDSTATE_NUMBER_OF_PARAMETERS - i - 1))) * getWorldStateParameterValueByIndex(&currentState, i);
        }

        //nasli smo možen startpoint, poišči se endpoint, kamor se povezuje akcija
        WorldStateGraphConnection newConnection;
        newConnection.connectingAction = actionFromActionList;
        newConnection.endpointNodeIndex = findEndpointNodeIndex(&adjacencyList[numericNodeIndex], actionFromActionList);
        adjacencyList[numericNodeIndex].adjacentVerticesLinks.Add(newConnection);
    }
    else if (currentParameterValue < 2) {
        setWorldStateParameterValueByIndex(&currentState, propertyIndex, currentParameterValue);
        FindWorldStateStartpointsForActions(actionFromActionList, propertyIndex + 1, currentState);
    }
    else {
        setWorldStateParameterValueByIndex(&currentState, propertyIndex, 0);
        FindWorldStateStartpointsForActions(actionFromActionList, propertyIndex + 1, currentState);

        setWorldStateParameterValueByIndex(&currentState, propertyIndex, 1);
        FindWorldStateStartpointsForActions(actionFromActionList, propertyIndex + 1, currentState);
    }
}
```

Slika 3.38: Metoda FindWorldStateStartpointsForActions

3.5.1.3 Dijkstrin algoritem

Algoritem je leta 1956 razvil Edsger W. Dijkstra [30] in sodi med široko uporabljane algoritme za iskanje najkrajše poti od začetnega vozlišča do ostalih vozlišč v grafu. Deluje na kateremkoli povezanem grafu (z usmerjenimi ali neusmerjenimi povezavami), kjer so vrednosti povezav nenegativna števila. Uporablja prednostno vrsto (angl. *Priority Queue*) neobiskanih vozlišč, ki jo sproti posodablja. Za vsako vozlišče shranjuje, kako dolga je najkrajša pot do njega iz začetnega vozlišča. Pred začetkom algoritma moramo to vrednost v izhodiščnem vozlišču postaviti na 0 (izhodiščno vozlišče dodamo v prednostno vrsto), pri vseh ostalih pa na ∞ . V prednostni vrsti poišče vozlišče z najkrajšo potjo od začetnega, od koder poišče vse njegove še neobiskane sosedne. Če najdenega soseda še ni v prednostni vrsti, ga vanjo

doda. Za vsakega soseda preveri, če je seštevek najkrajše poti od izhodišča do izbranega vozlišča in povezave od vozlišča do soseda krajši od vrednosti najkrajše poti, ki jo hrani sosed. Če je temu tako, posodobi sosedovo vrednost najkrajše poti (da lahko pot do cilja po zaključku algoritma rekonstruiramo, moramo v vsakem vozlišču beležiti, iz katerega vozlišča smo ga dosegli). Ko preveri vse neobiskane sosede vozlišča, ga odstrani iz prednostne vrste. Tako nadaljuje, dokler ne doseže izbranega cilja oziroma, dokler prednostna vrsta ni prazna. Algoritem je primer požrešne metode, časovna zahtevnost originalne implementacije iz leta 1959 je $O(m + n^2)$, ob uporabi dvojiške kopice pa jo izboljšamo na $O((m + n) * \log(n))$. Teoretično lahko s Fibonaccijevo kopico implementacijo še pohitrimo, razen pri redkih grafih, kot je naš (2048 vozlišč in 2336 usmerjenih povezav) [31].

Pri naši implementaciji se algoritem izvede ob klicu metode *GOAPExecuteDijkstra* iz razreda *Blueprint*, ki kot parametre sprejme logične vrednosti, s katerimi opišemo trenutno stanje sveta v igri. Iz parametrov sestavi *WorldState*, za katerega na enak način kot metoda *FindWorldStateStartpointsForActions* poišče pripadajoče vozlišče v grafu, ki bo predstavljalo izhodišče za izvedbo Dijkstrinega algoritma. Kazalec na to vozlišče shrani v razredno spremenljivko *thisActorsDijkstraStartNode*, nato kliče metodo *prepareDijkstra* (slika 3.39), ki pripravi vsa vozlišča v grafu na izvedbo Dijkstrinega algoritma. Povezave v našem grafu so akcije, ki jih mora akter izvesti za prehod iz enega stanja v drugega. Ker se cene povezav spreminjajo (predvsem v odvisnosti od razdalje med akterjem in njegovimi interesnimi točkami) in ker bi radi na enostaven način urejali cene povezav in s tem vplivali na obnašanje akterjev, so v razredih, ki določajo akcije, definirane funkcije, ki vračajo ceno povezave.

```
void ACPG_GoapPlanner::prepareDijkstra() {
    for (int i = 0; i < adjacencyList.Num(); i++) {
        adjacencyList[i].dijkstraShortestPathFromStart = GOAP_MAX_INT_VALUE;
        adjacencyList[i].dijkstraPreviousNode = nullptr;
        adjacencyList[i].dijkstraVisited = false;
        adjacencyList[i].dijkstraDiscovered = false;
        adjacencyList[i].nodeIndexInBinaryHeapArray = 0;
    }

    thisActorsDijkstraStartNode->dijkstraShortestPathFromStart = 0;
    thisActorsDijkstraStartNode->dijkstraPreviousNode = thisActorsDijkstraStartNode;
}
```

Slika 3.39: Metoda *prepareDijkstra*

Ko so vsa vozlišča pripravljena, se kliče metoda *performDijkstra* (slika 3.40), ki najprej ustvari prednostno vrsto in jo shrani v spremenljivko *priorityQueue*. Vanjo doda kazalec na začetno vozlišče, ki ga najde v razredni spremenljivki *thisActorsDijkstraStartNode*. Potem vstopi v neskončno zanko, ki se prekine, ko je prednostna vrsta prazna. Iz prednostne vrste vzame kazalec na vozlišče z najmanjšo vrednostjo in ga shrani v spremenljivko *currentNode*. Potem preišče vse povezave v *currentNode*. Za vsako povezavo preveri, ali je sosed, kamor povezava vodi, še neobiskan. Če je temu tako, shrani seštevek cene poti od začetka do *currentNode* in cene poti od *currentNode* do sosedu (dobi jo s klicem metode *getPerformCost* objekta *connectingAction*, kjer kot parametra posreduje *WorldState* trenutnega vozlišča in logično vrednost, ki je resnična, če je *currentNode* trenutno stanje sveta v igri. Tako bo metoda lahko vrnila ustrezno ceno poti glede na vozlišče, ki ga preiskujemo). Če je cena poti ugodnejša od vrednosti, ki jo hrani sosedno vozlišče, posodobimo njegovo ceno poti, nastavimo njegov kazalec *dijkstraPreviousNodeConnectingAction* na akcijo, ki ga povezuje s *currentNode*, predhodno vozlišče nastavimo na *currentNode* in posodobimo njegovo mesto v prednostni vrsti, če se v njej nahaja (če je vrednost *dijkstraDiscovered* enaka *true*). Če smo sosedu odkrili prvič, mu nastavimo *dijkstraDiscovered* na *true* in ga dodamo v prednostno vrsto. Ko pregledamo vse sosede vozlišča *currentNode*, ga odstranimo iz prednostne vrste in označimo kot obiskanega.

```

void ACPG_GoapPlanner::performDijkstra() {
    //ustvari priority queue s samo zacetnim vozliscom
    PriorityQueue priorityQueue = PriorityQueue();
    priorityQueue.insertNewNode(thisActorsDijkstraStartNode);

    while (true) {
        //najdi neobiskano vozlisce z najmanjso razdaljo od zacetnega vozlisca
        if (priorityQueue.isEmpty()) {
            //obiskali smo vsa vozlisca, ki se jih da obiskati
            break;
        }

        WorldStateGraphNode* currentNode = priorityQueue.getMinDistanceNode();

        //obisci neobiskane sosedne od currentNode
        for (int i = 0; i < currentNode->adjacentVerticesLinks.Num(); i++) {
            Action* connectingAction = currentNode->adjacentVerticesLinks[i].connectingAction;
            int neighbourNodeIndex = currentNode->adjacentVerticesLinks[i].endpointNodeIndex;

            if (adjacencyList[neighbourNodeIndex].dijkstraVisited == false) {
                int newDistanceToNeighbour = currentNode->dijkstraShortestPathFromStart +
                    connectingAction->getPerFormCost(&currentNode->worldState, (currentNode == thisActorsDijkstraStartNode));

                if (newDistanceToNeighbour < adjacencyList[neighbourNodeIndex].dijkstraShortestPathFromStart) {
                    adjacencyList[neighbourNodeIndex].dijkstraShortestPathFromStart = newDistanceToNeighbour;
                    adjacencyList[neighbourNodeIndex].dijkstraPreviousNodeConnectingAction = connectingAction;
                    adjacencyList[neighbourNodeIndex].dijkstraPreviousNode = currentNode;
                    if (adjacencyList[neighbourNodeIndex].dijkstraDiscovered == true) {
                        priorityQueue.fixHeapAtIndex(adjacencyList[neighbourNodeIndex].nodeIndexInBinaryHeapArray);
                    }
                }

                if (adjacencyList[neighbourNodeIndex].dijkstraDiscovered == false) {
                    adjacencyList[neighbourNodeIndex].dijkstraDiscovered = true;
                    priorityQueue.insertNewNode(&adjacencyList[neighbourNodeIndex]);
                }
            }
        }

        //nastavi currentNode na visited
        priorityQueue.deleteRootNode();
        currentNode->dijkstraVisited = true;
    }
}

```

Slika 3.40: Metoda performDijkstra

Po izvedbi metode *GOAPExecuteDijkstra* lahko iz razreda *Blueprint* kličemo metodo *GOAPGetKillPlayerCost*, ki najprej pripravi akterjev cilj, predstavljen s spremenljivko *goapGoal* tipa *WorldState*, v kateri so vse vrednosti parametrov privzete (enake 2), razen parametra *playerDead*, ki je enak 1. Potem se kliče metoda *evaluateGoal*, ki na podoben način, kot metoda *findWorldStateStartpointsForActions* rekurzivno preišče vsa vozlišča v seznamu *adjacencyList*, ki danemu cilju ustrezajo in med njimi poišče vozlišče z najbolj ugodno ceno poti. Indeks najugodnejšega cilja v seznamu *adjacencyList* shrani v razredno spremenljivko *bestGoalIndexInAdjacencyList*, ceno pa shrani v *bestGoalExecutionCost*. Če cilj iz trenutnega stanja sveta ni dosegljiv, v spremenljivki *bestGoalIndexInAdjacencyList* shrani vrednost -1.

Metoda *GOAPGetKillPlayerCost* nato kliče metodo *outputBestPath* (slika 3.41), ki v konzolo izpiše ime akterja in ceno najboljše poti, ki jo lahko izvede. Če cilja iz

trenutnega stanja ni mogoče doseči, to ustrezno izpiše v konzoli. Pot rekonstruira od najugodnejšega ciljnega vozlišča proti izhodišču (torej prvo akcijo, ki jo bo akter izvedel v izpisu najdemo na dnu, zadnjo pa na vrhu), pri čemer v konzolo izpisuje ima akcije, ki se morajo izvesti in ceno izvedbe. Ob koncu metode kazalec do naslednje akcije, ki jo mora akter izvesti, najdemo v razredni spremenljivki *nextActionToExecute*. Izpis načrta GOAP je viden na sliki 3.42.

```
void ACPG_GoapPlanner::outputBestPath() {
    nextActionToExecute = defaultActionToExecute;

    UE_LOG(LogTemp, Warning, TEXT("%s - BEST GOAL EXECUTION COST: %i"), *this->GetName(), bestGoalExecutionCost);

    if (bestGoalIndexInAdjacencyList == -1) {
        UE_LOG(LogTemp, Warning, TEXT("Cannot reach the selected goal from current world state"));
    }
    else {
        WorldStateGraphNode* nodeToOutput = &adjacencyList[bestGoalIndexInAdjacencyList];
        while (true) {
            if (nodeToOutput == nodeToOutput->dijkstraPreviousNode) {
                break;
            }

            nextActionToExecute = nodeToOutput->dijkstraPreviousNodeConnectingAction;
            FString outputActionName = nodeToOutput->dijkstraPreviousNodeConnectingAction->name;
            int outputActionPerformCost = nodeToOutput->dijkstraPreviousNodeConnectingAction->getPerformCost(&nodeToOutput->dijkstraPreviousNode->worldState,
                (nodeToOutput->dijkstraPreviousNode == thisActorsDijkstraStartNode));
            UE_LOG(LogTemp, Warning, TEXT("%s - %i"), *outputActionName, outputActionPerformCost);

            nodeToOutput = nodeToOutput->dijkstraPreviousNode;
        }
    }
}
```

Slika 3.41: Metoda outputBestPath

```
BP_EnergySoldier_C_UAID_74563C2971A4C3F701_1629467765 - BEST GOAL EXECUTION COST: 3929
Bow Attack - 3000
Move to Player's Bow Range - 0
Equip Bow - 1
Move to the High Ground - 928
```

Slika 3.42 Izpis načrta GOAP, prva akcija je na dnu, zadnja pa na vrhu izpisa

Metoda *GOAPGetKillPlayerCost* vrača spremenljivko *bestGoalExecutionCost*, pomnoženo z *goalCostMultiplier*, ki ga dobi kot parameter ob klicu iz razreda Blueprint. Zaenkrat je pokončati nasprotnika akterjev edini cilj. Ker Dijkstrin algoritem poišče poti do vseh vozlišč, dosegljivih iz trenutnega stanja sveta, bi cilje lahko brez težav dodajali. Pri tem bi ceno poti do vsakega pomnožili z njegovim multiplikatorjem (ki bi predstavljal prioriteto cilja), akter pa bi skušal doseči tistega z najbolj ugodnim zmnožkom. Ko metoda vrne zmnožek, lahko iz razreda Blueprint kličemo metodo *GOAPTriggerNextActionBlueprintEvent*, ki v akciji, na katero kaže kazalec *nextActionToExecute*, sproži metodo *triggerBlueprintEvent*. Ob tem se sproži ustrezen dogodek v razredu Blueprint akterja, v katerem je definiran potek izvedbe dane akcije.

3.5.1.4 Prednostna vrsta

Dijkstrin algoritem smo izboljšali z uvedbo prednostne vrste, predstavljene z minimalno binarno kopico (angl. *Minimum Binary Heap*), podatkovno strukturo, ki je urejena kot binarno drevo, v katerem je vrednost starševskega vozlišča vedno manjša od vrednosti v njegovih otrocih. Zato se vozlišče z najmanjšo vrednostjo zmeraj nahaja v korenu drevesa. Najpogostejše operacije, ki jih izvajamo nad binarno kopico so vstavljanje in brisanje vozlišča, pridobivanje najmanjšega vozlišča in vzdrževanje kopice (angl. *reheapify*) [32].

Pri operaciji vstavljanja novo vozlišče dodamo na konec kopice, na prvo prosto mesto na dnu binarnega drevesa. Če je vstavljeno vozlišče manjše od njegovega starša, ga z njim zamenjamo. Ta korak ponavljamo, dokler podatkovna struktura ne izpolnjuje lastnosti minimalne binarne kopice, torej, dokler ni vrednost starša manjša od vrednosti vstavljenega vozlišča. Če ima novo vozlišče manjšo vrednost od ostalih v kopici, bo s ponavljanjem tega koraka prišlo v koren drevesa. Časovna zahtevnost operacije je $O(\log(n))$, kjer je n število vozlišč v kopici [31]. Ko Dijkstrin algoritem vozlišču, ki se že nahaja v kopici, zmanjša vrednost, jo uredimo enako kot pri operaciji vstavljanja, le da urejanje začnemo v vozlišču, ki smo mu spremenili vrednost, nato pa kopico urejamo navzgor, proti korenu drevesa. Urejanja navzdol ne potrebujemo, saj bosta ob zmanjšanju vrednosti vozlišča vrednosti njegovih otrok ostali večji, Dijkstin algoritem pa vozlišču nikoli ne bo povečal vrednosti.

Ko Dijkstrin algoritem nastavi še neobiskano vozlišče z najmanjšo vrednostjo (torej vozlišče v korenu kopice) na obiskano, ga mora odstraniti iz kopice. Pri tej operaciji koren zamenjamo z zadnjim vozliščem kopice (najbolj desno v spodnjem nivoju drevesa). Na tem mestu je treba preveriti, če drevo z novim korenom še vedno ustreza lastnostim minimalne binarne kopice. Koren drevesa primerjamo z obema otrokoma. Če vrednost v korenu ni manjša od vrednosti v otrocih, zamenjamo koren z otrokom z manjšo vrednostjo. To ponavljamo, dokler ni vozlišče na ustreznem mestu v kopici. Časovna zahtevnost operacije je $O(\log(n))$.

Operacija pridobivanja najmanjšega vozlišča vrne vozlišče z najmanjšo vrednostjo v kopici. Ker v dvojiškem drevesu sproti vzdržujemo lastnosti minimalne dvojiške kopice, je iskano vozlišče zmeraj v korenu drevesa. Časovna zahtevnost operacije je zato zelo ugodna, $O(1)$ [32].

Prav tako se uporablja operacija iskanja po kopici, kjer bi morali v najslabšem primeru pregledati celotno kopico. Časovno potratni operaciji iskanja smo se popolnoma izognili, saj pri vsakem vozlišču v grafu (seznam *adjacencyList*) beležimo tudi njegov položaj v kopici. Tako smo časovno zahtevnost iskanja spremenili v $O(1)$.

Prednostno vrsto smo predstavili z razredom *PriorityQueue* (slika 3.43). Razred sestavljajo seznam kazalcev na vozlišča v grafu in metode, potrebne za vzdrževanje minimalne binarne kopice, s katero je prednostna vrsta realizirana. Kopica je predstavljena s spremenljivko *minBinaryHeapArray*, ki hrani seznam kazalcev na vozlišča v grafu *adjacencyList*.

```
class PriorityQueue {
private:
    TArray<WorldStateGraphNode*> minBinaryHeapArray;
public:
    void fixHeapAtIndex(int heapArrayIndex);
    bool heapIsEmpty();
    WorldStateGraphNode* getMinDistanceNode();
    void insertNewNode(WorldStateGraphNode* newNode);
    void deleteRootNode();

    PriorityQueue();
};
```

Slika 3.43: Razred PriorityQueue

Najprej smo implementirali metodo *fixHeapAtIndex* (slika 3.44), ki prejme indeks elementa (*heapArrayIndex*) v seznamu *minBinaryHeapArray*, nato pa vstopi v neskončno zanko. Za vozlišče, ki se v seznamu nahaja na indeksu *heapArrayIndex*, poišče indeks starševskega vozlišča v seznamu po enačbi $(heapArrayIndex - 1) / 2$, pri čemer mora biti dobljena vrednost zaokrožena navzdol. Če je vrednost, ki se uporablja za urejanje kopice (vrednost spremenljivke *dijkstraShortestPathFromStart*, kjer Dijkstrin algoritem beleži najkrajšo pot do vozlišča iz začetnega vozlišča) v staršu večja,

kot v vozlišču, določenem s *heapArrayIndex*, mora metoda vozlišči v kopici zamenjati. Pri tem pripadajočima objektoma *WorldStateGraphNode* posodobi parameter *nodeIndexInBinaryHeapArray*, da bo lahko Dijkstrin algoritem za vsako vozlišče v grafu stanj sveta še naprej enostavno poiskal njegovo mesto v kopici. V nasprotnem primeru dvojiško drevo ustreza lastnostim minimalne binarne kopice, zato zapustimo zanko.

```
void PriorityQueue::fixHeapAtIndex(int heapArrayIndex)
{
    //metoda poskrbi, da so vrednosti dijkstraShortestPathFromStart pri starsih vedno manjše kot pri otrocih
    //navzdol ni treba preverjat, ker lahko Dijkstra vozliscem vrednost le zmanjsuje
    while (true) {
        int parentNodeIndex = (heapArrayIndex - 1) / 2;
        if (this->minBinaryHeapArray[heapArrayIndex]->dijkstraShortestPathFromStart <
            this->minBinaryHeapArray[parentNodeIndex]->dijkstraShortestPathFromStart) {
            //v starsu je dijkstraShortestPathFromStart vecja, torej zamenjaj starsa in otroka
            WorldStateGraphNode* swapStorage = this->minBinaryHeapArray[parentNodeIndex];
            this->minBinaryHeapArray[parentNodeIndex] = this->minBinaryHeapArray[heapArrayIndex];
            this->minBinaryHeapArray[heapArrayIndex] = swapStorage;

            this->minBinaryHeapArray[parentNodeIndex]->nodeIndexInBinaryHeapArray = parentNodeIndex;
            this->minBinaryHeapArray[heapArrayIndex]->nodeIndexInBinaryHeapArray = heapArrayIndex;
        }
        else {
            break;
        }
    }
}
```

Slika 3.44: Metoda fixHeapAtIndex

Metoda *heapsIsEmpty* (slika 3.45) vrača logično vrednost *true* samo, če kopica vsebuje kakšno vozlišče. S klicem *getMinDistanceNode* (slika 3.45) dobimo vozlišče v kopici z najmanjšo vrednostjo, ki ga v pravilno urejeni kopici vedno najdemo v prvem elementu seznama *minBinaryHeapArray*. Ko v kopico dodajamo novo vozlišče, uporabimo metodo *insertNewNode* (slika 3.45), ki novo vozlišče doda na konec seznama *minBinaryHeapArray*, nato pa kopico uredi z metodo *fixHeapAtIndex*, kjer kot parameter pošlje indeks na novo dodanega vozlišča.

```

bool PriorityQueue::heapIsEmpty()
{
    return (this->minBinaryHeapArray.Num() == 0);
}

WorldStateGraphNode* PriorityQueue::getMinDistanceNode()
{
    return (this->minBinaryHeapArray[0]);
}

void PriorityQueue::insertNewNode(WorldStateGraphNode* newNode)
{
    this->minBinaryHeapArray.Add(newNode);
    fixHeapAtIndex(this->minBinaryHeapArray.Num() - 1);
}

```

Slika 3.45: Metode `heapIsEmpty`, `getMinDistanceNode` in `insertNewNode`

Ko Dijkstrin algoritem obiše neobiskano vozlišče z najmanjšo vrednostjo (torej vozlišče, ki se nahaja v korenu kopice), ga mora iz prednostne vrste odstraniti. Pri tem uporabi metodo *deleteRootNode* (slika 3.46), ki najprej zamenja koren kopice z vozliščem, ki se na dnu binarnega drevesa nahaja najbolj desno (se pravi vozlišče na koncu seznama *minBinaryHeapArray*), potem pa spodnje desno vozlišče odstrani iz seznama. Pri tem lastnosti kopice navadno ni več zadoščeno, zato mora iz korena drevesa navzdol preverjati, če sta otroka manjša od starša. Če je temu tako, je lastnostim kopice zadoščeno, zato lahko s preverjanjem zaključimo. V nasprotnem primeru v seznamu zamenjamo starša z otrokom z najmanjšo vrednostjo in zaključimo preverjanje, če otrok ni več mogoče najti (kadar je *terminateWhileLoop* enak *true*).

```

void PriorityQueue::deleteRootNode()
{
    //odstrani root in ga zamenjaj z najbolj desnim elementom v arrayu
    this->minBinaryHeapArray[0] = this->minBinaryHeapArray[this->minBinaryHeapArray.Num() - 1];
    this->minBinaryHeapArray.RemoveAt(this->minBinaryHeapArray.Num() - 1);

    //za starsa pogledj ce imata otroka vecji dijkstraShortestPathFromStart. Ce temu ni tako, ga zamenjaj z manjsim izmed obeh otrok.
    int currentParentNodeIndex = 0;
    while (true) {
        if (heapIsEmpty()) {
            break;
        }

        int currentMinValue = this->minBinaryHeapArray[currentParentNodeIndex]->dijkstraShortestPathFromStart;
        int nodeWithMinValueIndex = currentParentNodeIndex;

        //poglej, katero izmed 3 vozlic (stars in 2 otroka) ima najmanjsi dijkstraShortestPathFromStart
        bool terminateWhileLoop = false;
        for (int i = 1; i < 3; i++) {
            int childIndex = 2 * currentParentNodeIndex + i;
            if (childIndex >= minBinaryHeapArray.Num()) {
                //smo na dnu heapa na tem mestu pa ni otroka, torej nehaj preiskovati
                terminateWhileLoop = true;
                break;
            }
            if (this->minBinaryHeapArray[childIndex]->dijkstraShortestPathFromStart < currentMinValue) {
                currentMinValue = this->minBinaryHeapArray[childIndex]->dijkstraShortestPathFromStart;
                nodeWithMinValueIndex = childIndex;
            }
        }

        //ce ima kateri od otrok manjso vrednost od starsa, ju zamenjaj in nadaljuj zanko. Sicer zakljuci zanko.
        if (nodeWithMinValueIndex == currentParentNodeIndex) {
            break;
        }

        WorldStateGraphNode* swapStorage = this->minBinaryHeapArray[currentParentNodeIndex];
        this->minBinaryHeapArray[currentParentNodeIndex] = this->minBinaryHeapArray[nodeWithMinValueIndex];
        this->minBinaryHeapArray[nodeWithMinValueIndex] = swapStorage;

        this->minBinaryHeapArray[currentParentNodeIndex]->nodeIndexInBinaryHeapArray = currentParentNodeIndex;
        this->minBinaryHeapArray[nodeWithMinValueIndex]->nodeIndexInBinaryHeapArray = nodeWithMinValueIndex;

        currentParentNodeIndex = nodeWithMinValueIndex;

        if (terminateWhileLoop) {
            break;
        }
    }
}

```

Slika 3.46: Metoda deleteRootNode

3.5.2 Uvoz animacij

Ko smo razvili algoritem GOAP, smo morali definirati obnašanje akterjev ob izvedbi posameznih akcij. Pri tem so ključnega pomena dobre animacije. Unreal Engine nam med drugim omogoča izdelavo lastnih animacij [33], seveda pa lahko animacije uvozimo iz specializiranih programov (prej smo opisali postopek uvoza pri programu Blender). Ker je izdelava animacij likov igre zelo zamudna in bi težko dosegli realizem, ki ga igralci od sodobnih iger pričakujejo, se v industriji filmov in iger uporablja tehnika zajema gibanja (angl. *Motion Capture*) [34]. Ker je tudi ta zamudna, predvsem pa draga, si lahko pomagamo s knjižnicami, ki že vsebujejo veliko na ta način ustvarjenih animacij.

Uporabili smo brezplačno knjižnico *Adobe Mixamo* [13]. Animacije iz knjižnice smo prenesli (angl. *animation retargeting*) na privzet Unrealov lik *Quinn*, ki ga uporabljamo v naši igri, z uporabo brezplačnega programa *Mixamo Converter* [14]. Za predvajanje animacij smo v razredih Blueprint akterjev uporabili vozlišče *Play Montage* [35], zato smo morali uvožene gradnike *Animation Sequence* pretvoriti v *Animation Montage* (na gradniku izberemo *desni klik*→*Create*→*Create Anim Montage*).

3.5.3 Določanje poteka igre z Blueprint

Obnašanje akterjev ob izvedbi posameznih akcij in obnašanje lika, ki ga nadzoruje igralec, smo implementirali v programskem jeziku Blueprint. Za osnovo smo kopirali Unrealov razred *BP_ThirdPersonCharacter* in kopijo poimenovali *BP_Soldier1*. *BP_ThirdPersonCharacter* deduje iz razreda *Character*, zato smo v urejevalniku razredu Blueprint *BP_Soldier1* spremenili starševski razred (*File*→*Reparent Blueprint*) na *CPP_GoapPlanner* (ki ravno tako deduje iz razreda *Character*). Tako smo lahko v razredu *BP_Soldier1* in v razredih, ki iz njega dedujejo, uporabili vse funkcionalnosti razreda *CPP_GoapPlanner*.

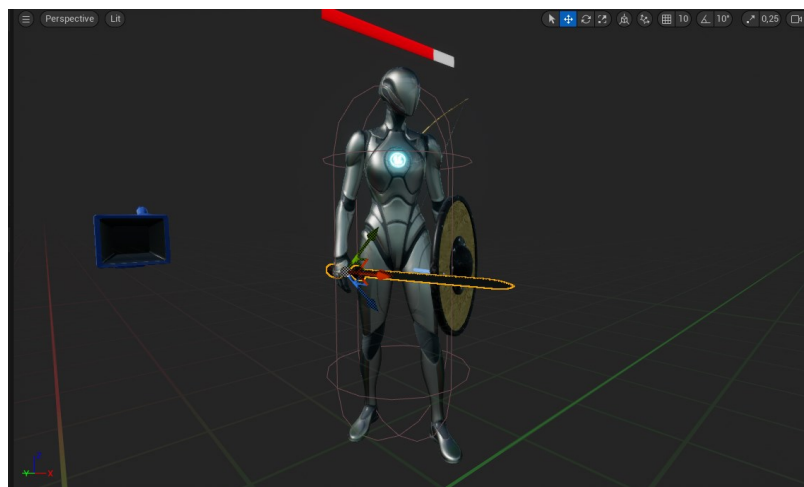
Potem smo ustvarili razreda *BP_FriendlySoldier* in *BP_EnemySoldier*, katerima smo kot starševski razred določili *BP_Soldier1*. Tako razreda poleg vse funkcionalnosti dedujeta tudi komponente, ki razred *BP_Soldier1* sestavljajo (na primer 3D modele). Ustvarili smo še razred *BP_Player*, ki deduje iz razreda *BP_FriendlySoldier*. Da bi igralec lahko nadzoroval instanco razreda *BP_Player*, smo v razredu *BP_ThirdPersonGameMode* pri parametru *Default Pawn Class* izbrali razred *BP_Player*.

Da bi bil nad akterji na sceni prikazan indikator njihovega zdravja, smo ustvarili *Widget Blueprint* z imenom *WB_HealthBar*. V njem smo v komponento *Canvas Panel* dodali komponento tipa *Progress Bar* in jo poimenovali *HealthBar*. V njenih nastavitvah smo pri *Anchor* izbrali sredino zaslona in obkljukali možnost *Is Variable*, da bi v programskem jeziku Blueprint lahko dobili referenco do komponente in ji spreminjali lastnosti.

Na podoben način smo ustvarili razred *WB_WidgetOverlay*, ki bo igralcu med igro prekrival zaslon. Ustvarili smo komponento *HealthBar*, v kateri bo igralec lahko spremljal svoje zdravje in jo pomaknili v spodnji levi kot zaslona. Prav tako smo dodali komponento tipa *Border*, ki bo igralcu pomagala pri merjenju z lokom, jo poimenovali *Crosshair* in postavili na sredo zaslona. Ker želimo, da je *Crosshair* prikazan le takrat, ko ima igralec opremljen lok, smo tudi tu obkljukali *Is Variable*.

Preden smo začeli z implementacijo logike, smo liku v razredu *BP_Soldier1* dodali meč, lok in ščit. Ker so bili modeli iz Blenderja sestavljeni iz več objektov, smo najprej označili vse objekte, ki model sestavljajo, nato pa jih z *Tools*→*Merge Actors* pretvorili v enoten gradnik tipa *Static Mesh*. Lok je bil že uvožen kot objekt tipa *Skeletal Mesh*, zato pretvarjanje ni bilo potrebno. Nato smo na ogrodje lika dodali točke pritrditve (angl. *Sockets*) tako za uporabljena, kot tudi za pospravljena orožja.

Potem smo v urejevalniku razreda *BP_Soldier1* na komponento *CollisionCylinder* dodali *WB_HealthBar*, na *CharacterMesh0* pa komponente orožja (gradnike iz razdelka *Content Browser* smo preprosto zapeljali na izbrano starševsko komponento razreda *BP_Soldier1*). Novo ustvarjenim komponentam smo spremenili ime, komponentam orožja pa ustrezno nastavili parameter *Parent Socket*. Videz 3D komponent razreda je prikazan na sliki (slika 3.47).

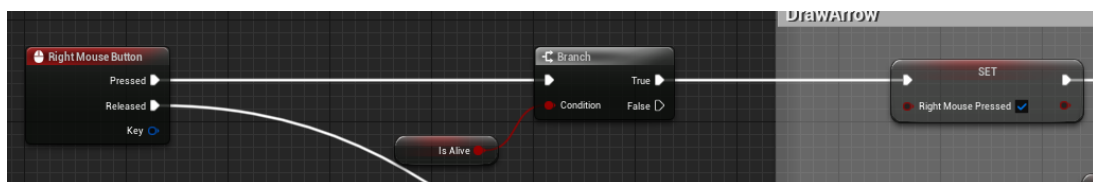


Slika 3.47: Razred *BP_Soldier1*

3.5.3.1 Streljanje z lokom

Da bi bile vse akcije, ki smo jih definirali v algoritmu GOAP, ustrezno izvedene v 3D svetu naše igre in da bi se lik, ki ga nadzoruje igralec, odzival na pritisk tipk (E – menjava orožja; desni klik – ščit, merjenje z lokom; levi klik – udarec z mečem, izstrelitev puščice ob napetem loku), smo predvsem znotraj razreda *BP_Soldier1* in nekoliko tudi v izpeljanih razredih definirali precej dogodkov in funkcij. Za ilustracijo delovanja programskega jezika Blueprint si oglejmo, kako poteka streljanje z lokom pri igralcu (inteligentni akterji uporabljajo podobno logiko). Vse navedene spremenljivke so javne razredne spremenljivke.

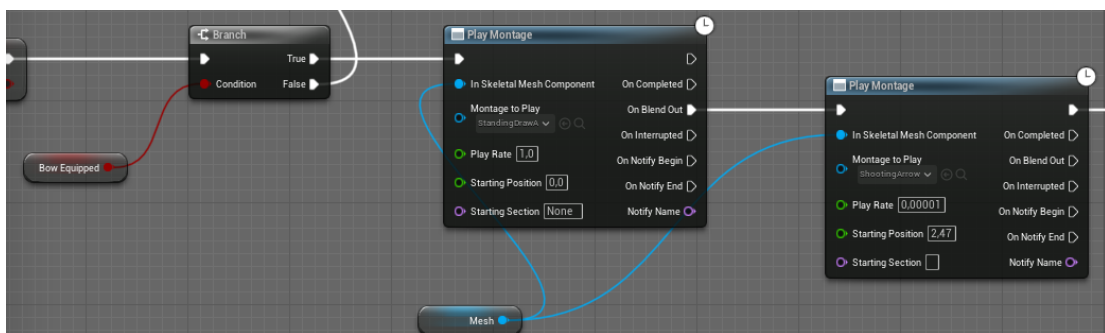
Pri dogodkih, povezanih s pritiskom desnega klika, se proži vozlišče *RightMouseButton* (slika 3.48). Veja diagrama poteka, ki iz njega izhaja, se ob sprožitvi posameznega dogodka izvede le enkrat. Ob dogodku se lahko kliče vozlišče, vezano na izhod *Pressed* ob pritisku desnega klika oziroma *Released*, ko igralec spusti desni klik. Ko igralec pritisne desni klik, želimo, da začne njegov lik napenjati lok, zato si najprej oglejmo povezavo, ki izhaja iz izhoda *Pressed*. Najprej se v vozlišču *Branch* preveri vrednost logične spremenljivke *Is Alive*, ki hrani vrednost *true*, če je igralec živ. Če je vrednost spremenljivke *true*, se kliče vozlišče *Set*. V nasprotnem primeru se graf preneha izvajati (ob naslednji aktivaciji vozlišča *RightMouseButton* se bo spet izvedel od začetka). Ker je vrednost v vozlišču *Set* označena, se bo logična spremenljivka *RightMousePressed* v njem postavila na *true*.



Slika 3.48: Začetek grafa iz vozlišča *RightMouseButton*

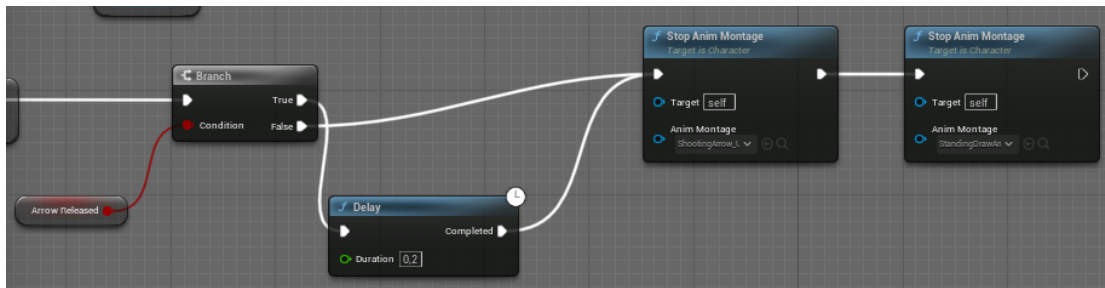
Nato preverjamo, ali je igralec opremljen z lokom oziroma s ščitom (slika 3.49). Če ima opremljen lok, se bo z vozliščem *PlayMontage* na komponenti *Mesh* (skeletalni 3D model lika v razredu *BP_Soldier1*) predvajala animacija napenjanja loka. Želen gradnik

Animation Montage smo izbrali na vhodu *Montage to Play*, referenco na komponento *Mesh* pa smo povezali na vhod *InSkeletalMeshComponent*. Ker želimo, da se animacija izvaja pri normalni hitrosti, je vrednost vhoda *PlayRate* enaka 1.0. Da bi se animacija predvajala od začetka, smo vrednost *StartingPosition* nastavili na 0.0. Vozlišče nam ponuja več izhodov. Ker bi radi nadaljevali z izvedbo grafa šele takrat, ko se animacija bliža koncu, smo na izhod *OnBlendOut* povezali novo vozlišče *PlayMontage*, v katerem predvajamo animacijo napenjanja loka s tako majhno vrednostjo *PlayRate*, da animacija praktično obstane na mestu. To vozlišče potrebujemo, ker želimo, da lok ostane napet tudi, če igralec za merjenje potrebuje več časa. Graf je naprej povezan iz prvega izhoda vozlišča *PlayMontage*, torej se bo animacija, ki jo sproži vozlišče predvajala medtem, ko se bo graf izvajal naprej.



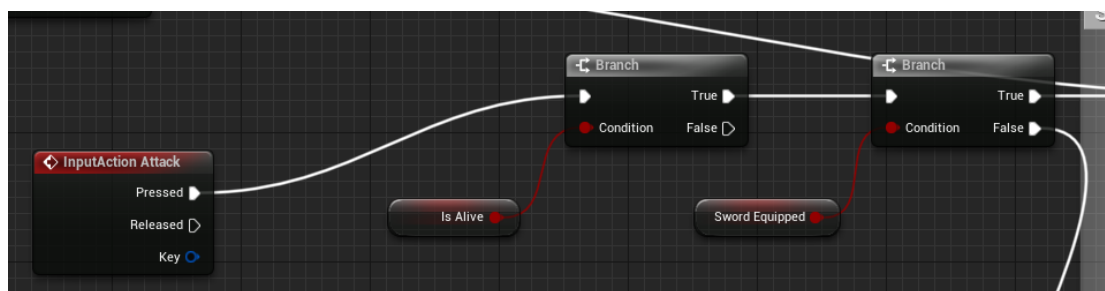
Slika 3.49: Uporaba vozlišč PlayMontage

Ko igralec spusti desni klik, želimo prekiniti izvajanje animacij, povezanih z napenjanjem loka in ustrezno nastaviti vrednosti spremenljivk, ki napenjanje loka urejajo. Da animacij ne bi prekinili takoj, ko igralec izstrelil puščico, preverimo, ali je igralec puščico že izstrelil (slika 3.50). V tem primeru z vozliščema *StopAnimMontage* prekinemo izvajanje morebitnih animacij. Sicer pred prekinjanjem počakamo 0.2 sekunde z uporabo vozlišča *Delay*.



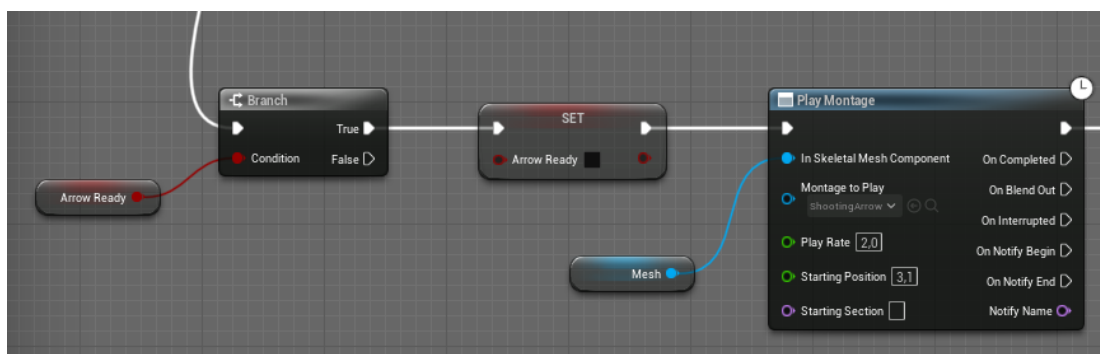
Slika 3.50: Uporaba vozlišč StopAnimMontage

Ob pritisku levega klika vozlišče *InputActionAttack* (slika 3.51) kliče vozlišče, vezano na izhod *Pressed*. Najprej se preveri, če je igralec sploh živ. Potem se preveri, če je opremljen z mečem. Če je temu tako, se izvede logika za napad z mečem, sicer sklepamo, da igralec trenutno uporablja lok.



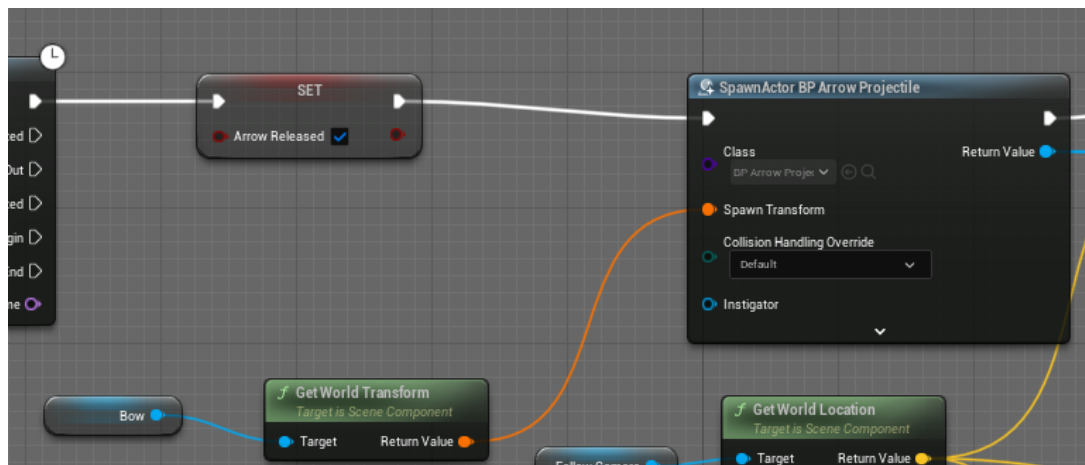
Slika 3.51: Dogodek ob pritisku levega klika

Z vozliščem *Branch* (slika 3.52) preverimo, ali je lok napet in pripravljen na izstrelitev puščice. Če je tako, nadaljujemo s predvajanjem animacije streljanja na razredni komponenti *Mesh*, spremenljivko *ArrowReleased* pa nastavimo na *true*.



Slika 3.52: Nadaljevanje grafa, ko je `SwordEquipped = false`

Z vozliščem *Spawn Actor from Class* (ki ob nastavljenem razredu spremeni ime) na sceni ustvarimo novo instanco razreda *BP_Arrow_Projectile*, kjer imamo 3D model puščice skupaj z njeno definicijo obnašanja. Puščico bi radi ustvarili tam, kjer se nahaja likov lok in jo zasukali tako, da je z njim poravnana. Zato smo vektor transformacij komponente *Bow* povezali v vhod *SpawnTransform* (slika 3.53).

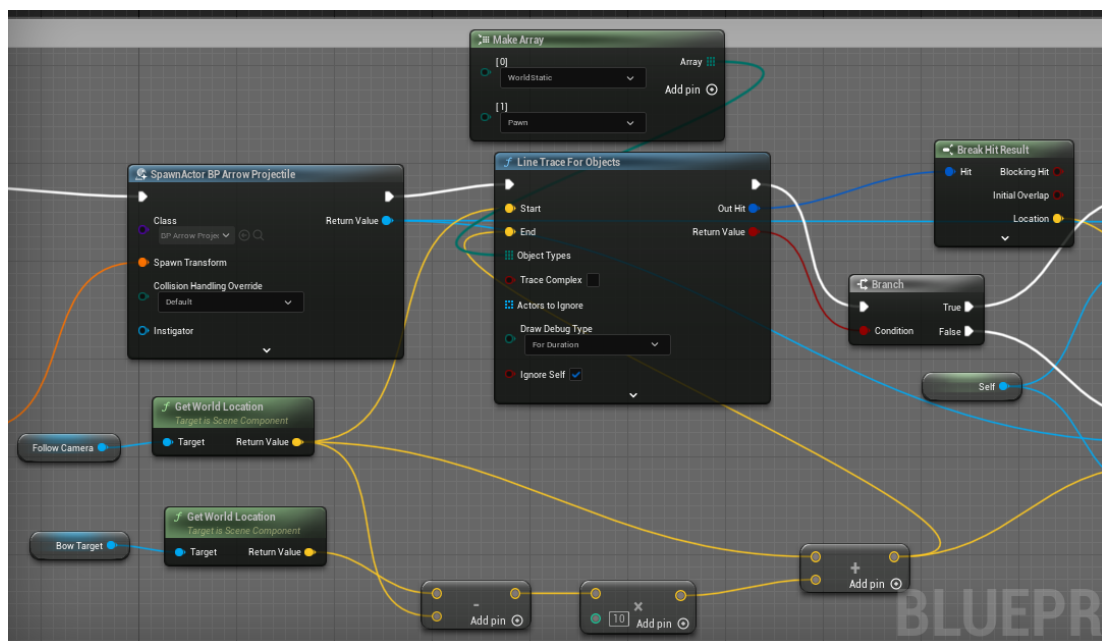


Slika 3.53: Uporaba vozlišča *SpawnActorFromClass*

Ko je puščica ustvarjena, ji moramo posredovati položaj tarče. V ta namen uporabimo vozlišče *LineTraceForObjects* (slika 3.54), ki v 3D prostoru naše igre ustvari daljico, katere začetno krajišče je določeno z vektorjem *Start*, končno pa z *End*. Pri tem poišče prvo presečišče daljice (iz krajišča *Start*) s katerimkoli objektom na sceni, ki ustreza tipom objekta, ki smo jih povezali v vhod *ObjectTypes*. Z vozliščem *MakeArray* (slika

3.54) smo ustvarili seznam možnih tipov, kjer smo dodali elementa *WorldStatic* za preverjanje trkov s statičnimi objekti (obzidje, tla, hiše, drevesa, ...) in *Pawn* za preverjanje trkov z akterji na sceni.

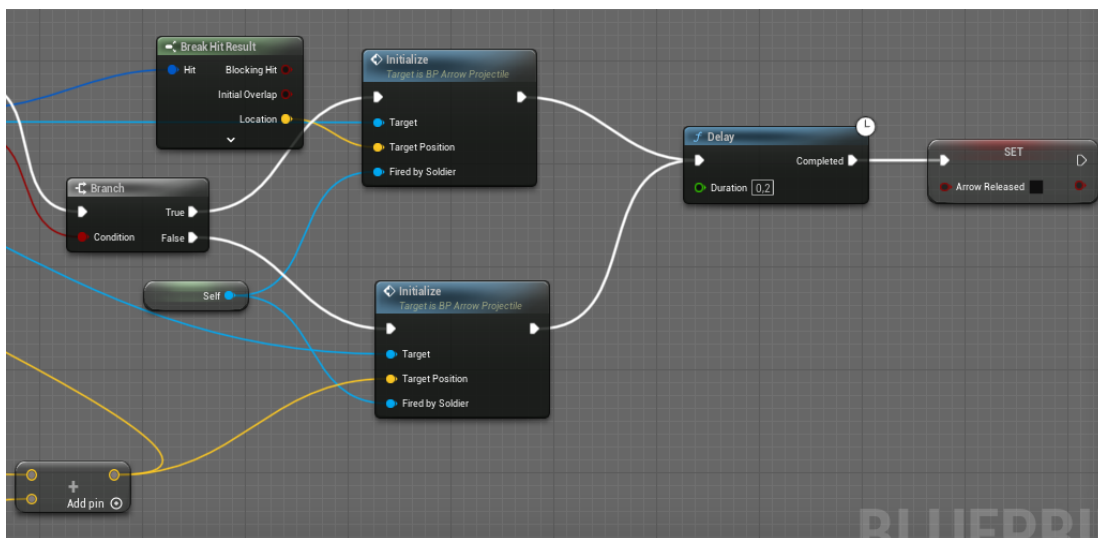
Pri merjenju z lokom igralec pričakuje, da bo puščica izstreljena v sredino njegovega pogleda, zato smo v vhod *Start* povezali položaj komponente *FollowCamera* (kamera, skozi katero igralec opazuje svet). Del razreda *BP_Soldier1* je tudi komponenta *BowTarget*, točka, ki je vedno postavljena pred kamero. Od njenega položaja smo odšteli položaj kamere, ga pomnožili s skalarjem 10, nato pa prišteli vektorju kamere. Dobljeni vektor smo povezali v vhod *End* in predstavlja končno točko daljice, za katero se ob merjenju računajo trki (slika 3.54).



Slika 3.54: Uporaba vozlišča LineTraceForObjects

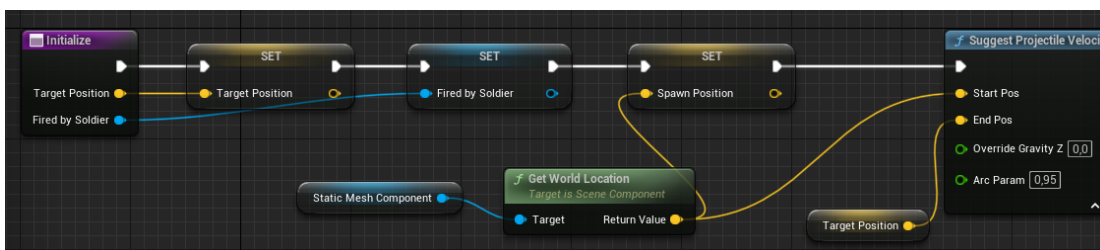
Če vozlišče najde trk, bomo podatkovno strukturo, ki predstavlja najden trk, našli na izhodu *OutHit*, logična vrednost izhoda *ReturnValue* pa bo *true*. Položaj trka najdemo tako, da strukturo *OutHit* razcepimo z vozliščem *BreakHitResult* (slika 3.54). Zatem kličemo funkcijo *Initialize* (slika 3.55), ki smo jo definirali v razredu *BP_Arrow_Projectile*. Ob klicu funkcije moramo poslati referenco instance razreda, kjer

se bo funkcija izvedla, zato smo na vhod *Target* povezali referenco ustvarjene puščice, ki jo dobimo na izhodu *ReturnValue* vozlišča *SpawnActorBP_Arrow_Projectile* (slika 3.54). Naša metoda *Initialize* pričakuje vhodna parametra *TargetPosition* in *FiredBySoldier*, kjer posredujemo položaj izbrane tarče in referenco na instanco razreda *BP_Soldier1*, torej vojaka, ki je puščico izstrelil (slika 3.55).



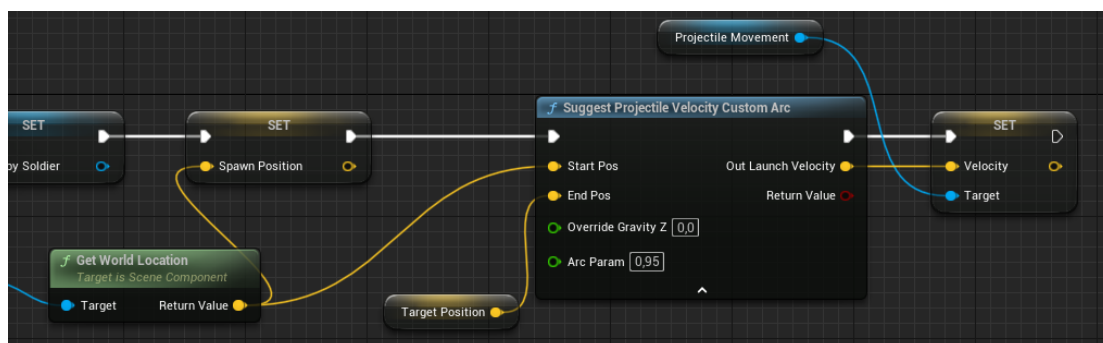
Slika 3.55: Klic funkcije Initialize iz razreda BP_Arrow_Projectile

V razredu *BP_ArrowProjectile* smo definirali prej omenjeno funkcijo *Initialize* (slika 3.56). Ob klicu funkcije se ustrezno nastavita razredni spremenljivki puščice *TargetPosition* in *FiredBySoldier* na vrednosti vhodnih parametrov, posredovanih ob klicu funkcije. Prav tako se v spremenljivko *SpawnPosition* zapiše trenutni položaj puščice.



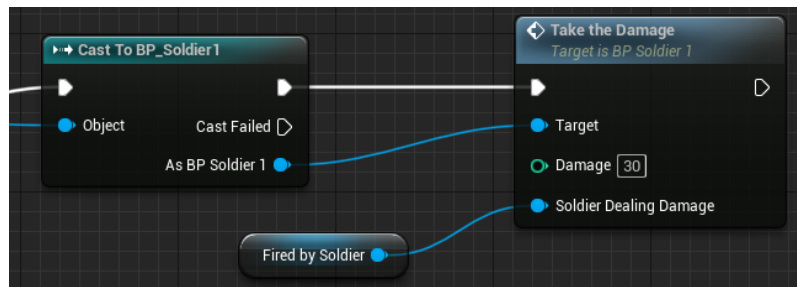
Slika 3.56: Definicija metode Initialize v razredu BP_ArrowProjectile

V razred smo dodali komponento *ProjectileMovement*, ki med drugim lahko izračuna pot izstrelka med začetno in končno lokacijo ob upoštevanju gravitacije. Z vozliščem *SuggestProjectileVelocityCustomArc* (slika 3.57) smo poiskali takšno začetno hitrost puščice, da bo potovala iz trenutne točke do izbrane tarče v želenem loku. Z vozliščem *Set* smo komponenti *ProjectileMovement* določili ustrezno začetno hitrost.



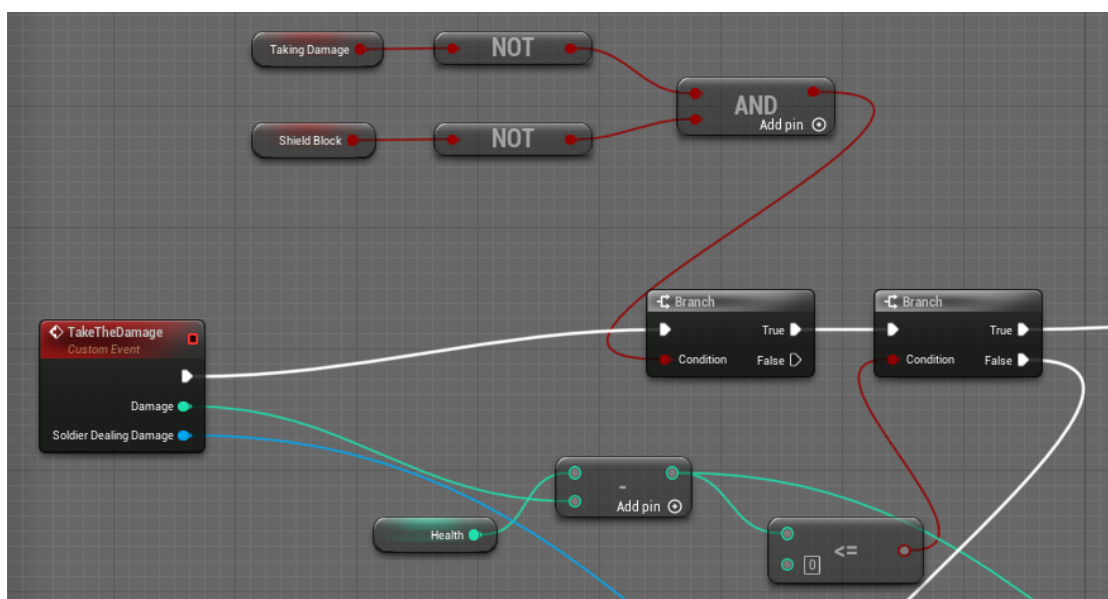
Slika 3.57: Uporaba vozlišča *SuggestProjectileVelocityCustomArc*

V razredu *BP_ArrowProjectile* smo uporabili vozlišče *EventTick*, ki se neprestano sproža. Tako lahko neprestano preverjamo morebitne trke puščice z akterji na sceni. Pri tem uporabljamo spremenljivko *LastPosition*, kjer hranimo položaj puščice ob zadnji aktivaciji vozlišča *EventTick*. Z vozliščem *LineTraceForObjects* nato preverjamo, če je med trenutno lokacijo puščice in lokacijo iz spremenljivke *LastPosition* prišlo do trka. Če je puščica trčila z statično oviro, puščico ustavimo in jo izbrišemo čez 10 sekund. V primeru, da je trčila z vojakom iz nasprotnega tabora kot vojak, ki je puščico izstrelil (če spremenljivka puščice *FiredBySoldier* hrani instanco razreda *BP_FriendlySoldier* in je vojak, ki ga je puščica zadela, instanca razreda *BP_EnemySoldier*, ali obratno), se pri zadetem vojaku kliče dogodek *TakeTheDamage* (slika 3.58), definiran v razredu *BP_Soldier1*.



Slika 3.58: Klic dogodka TakeTheDamage

Želimo, da je vojak ob pravočasni uporabi ščita varen pred udarcem in da tekom animacije reakcije na udarec ne prejema dodatnih udarcev. To rešimo v prvem vozlišču *Branch* ob klicu dogodka *TakeTheDamage*, v naslednjem pa ugotovimo, če je treba igralčevo spremenljivko *Health* postaviti na 0 (slika 3.59).



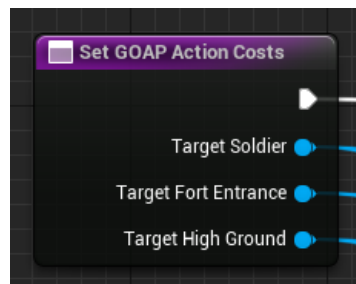
Slika 3.59: Definicija dogodka TakeTheDamage

3.5.3.2 Uporaba algoritma GOAP v razredu Blueprint

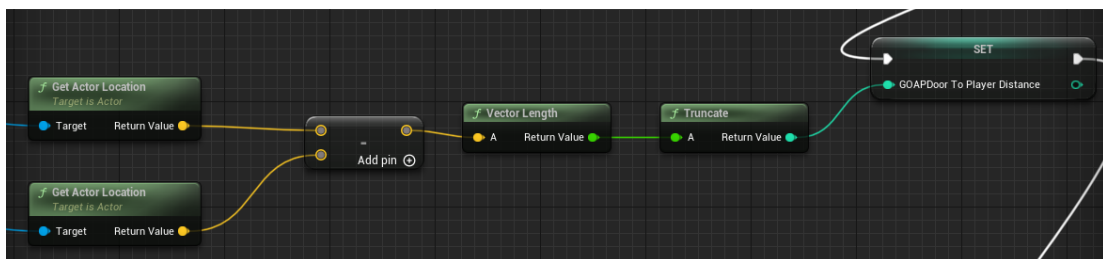
Ko se razred *BP_Soldier1* ustvari, se začene ponavljajoč časovnik, ki sproži dogodek *RunGoap* vsakih 0.05 sekunde. Vsak akter (entiteta, ki nadzoruje 3D lik, v tem primeru algoritem) v spremenljivki *AITargetSoldier* hrani referenco vojaka, ki je njegova trenutna tarča in *integer* spremenljivko *AIGoapFrequency*, od katere je odvisno število

izvedb algoritma GOAP. Algoritem se bo izvajal pogostejše, če se akter nahaja blizu igralca oziroma, če je igralec njegova trenutna tarča. Časovno zahtevnejše metode iz razreda *CPP_GoapPlanner* (na primer Dijkstrin algoritem), ob sprožitvi dogodka *RunGoap* v programski kodi Blueprint kličemo z verjetnostjo $1/AlGoapFrequency$, sicer čakamo na naslednjo aktivacijo dogodka *RunGoap*. Na ta način varčujemo s kapacitetami igralčevega računalnika.

V razredu *BP_Soldier1* smo implementirali funkcijo *SetGOAPActionCosts* (slika 3.60), s katero nastavimo vrednosti spremenljivk, za katere smo v programski kodi C++ omogočili urejanje v programski kodi Blueprint. Funkcija kot vhodne parametre prejme reference na trenutno tarčo, izbran vhod v obzidje in točko na višjem delu obzidja. Vrednost spremenljivke *GOAPDoorToPlayerDistance* na primer določi tako, da izračuna razdaljo med *TargetFortEntrance* in *TargetSoldier* (slika 3.61), nekaterim spremenljivkam pa vrednost nastavlja na vnaprej določeno število.

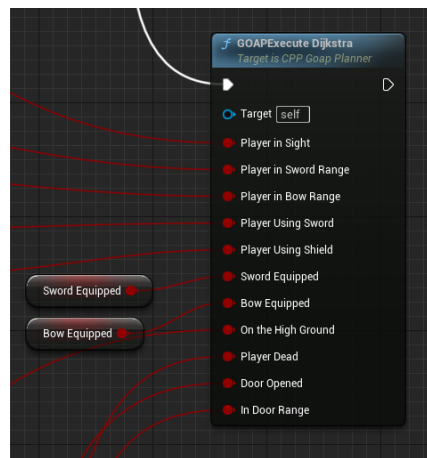


Slika 3.60: Začetno vozlišče funkcije SetGOAPActionCosts

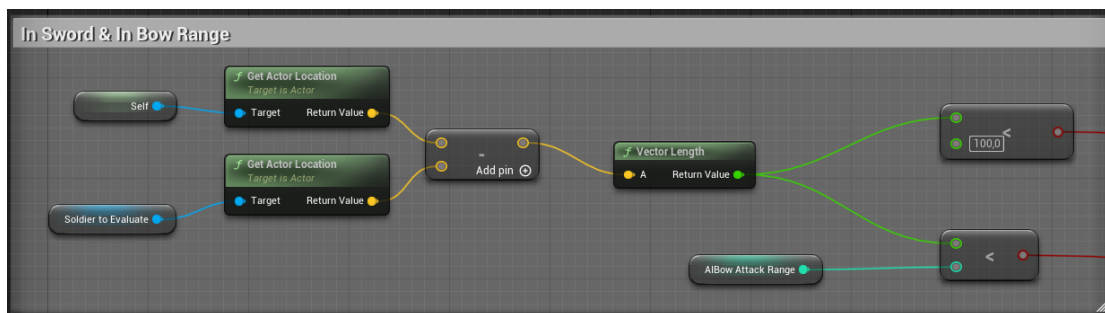


Slika 3.61: Nastavljanje spremenljivke GOAPDoorToPlayerDistance

V funkciji *PrepareAndExecuteDijkstraInGOAP*, ki kot parameter prejme referenco na tarčo (vhodni parameter *SoldierToEvaluate*), za katero želimo ob klicu metode ovrednotiti skupno ceno načrta GOAP, najprej kličemo funkcijo *SetGOAPActionCosts*, nato pa kličemo metodo *GOAPExecuteDijkstra*, kjer ustrezno nastavimo vhodne logične parametre, s katerimi opišemo trenutno stanje sveta (slika 3.62). Pri prikazanem primeru (slika 3.63) določamo vrednosti parametrov *PlayerInSwordRange* in *PlayerInBowRange*.



Slika 3.62: Klic metode GOAPExecuteDijkstra



Slika 3.63: Določanje vrednosti vhodnih parametrov

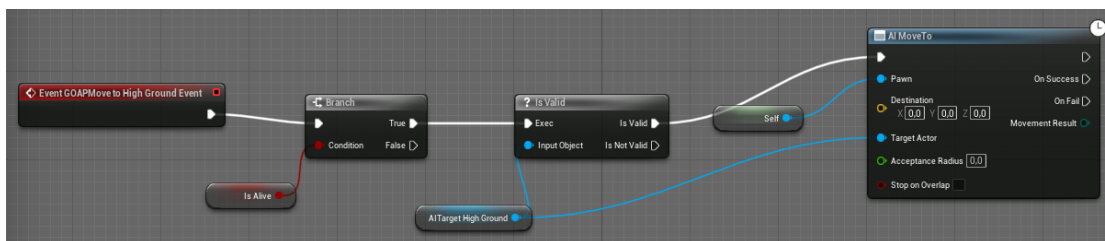
Opisane funkcije kličemo iz funkcij *CallNextGOAPAction* in *EvaluateAlternativeGOAPTargets*, ki se kličeta ob vsaki aktivaciji dogodka *RunGoap* (torej vsakih 0,05 sekunde). Obe funkciji ob klicu na prej opisan način uporabljata spremenljivko *AIGoapFrequency*.

CallNextGOAPAction najprej poišče akterju najbližji vhod v obzidje in najbližjo še nezasedeno točko na visokem delu obzidja (instance razreda *BP_HighGroundPosition*, postavljene po obzidju na sliki 3.64), iz katere se, če se le da, vidi akterjevo tarčo (*AITargetSoldier*). Nato kliče funkcijo *PrepareAndExecuteDijkstraInGOAP* in zapiše ceno načrta GOAP (z manjšimi popravki jo dobi s klicem metode *GOAPGetKillPlayerCost*) v razredno spremenljivko *AICurrentPlanCost* in sproži dogodek Blueprint za izvedbo naslednje akcije s klicem metode *GOAPTriggerNextActionBlueprintEvent*.

Da bi se akterji lahko navigirali po svetu, smo na nivo z uporabo gradnika *NavMeshBoundsVolume* dodali navigacijsko mrežo (na sliki 3.64 prikazana v zeleni barvi), na kateri akterji iščejo pot do zelenega cilja. Na sliki 3.65 je prikazana implementacija dogodka *GOAPMoveToHighGroundEvent*, ki se sproži ob klicu metode *GOAPTriggerNextActionBlueprintEvent*, implementirane v programskem jeziku C++ v razredu *ActionMoveToHighGround*. Uporabili smo vozlišče *AIMoveTo*, ki izračuna pot akterja do lokacije točke na visokem delu obzidja, referencirane v spremenljivki *AITargetHighGround* [36].



Slika 3.64: Navigacijska mreža in instance razreda *BP_HighGroundPosition*



Slika 3.65: Definicija dogodka GOAPMoveToHighGroundEvent

Funkcija *EvaluateAlternativeGOAPTargets* na podoben način, kot *CallNextGOAPAction* ovrednoti ceno načrta GOAP za uboj vojakov, referenciranih v razrednih spremenljivkah *AI SoldierDealingDamage* (kamor se shrani referenca nasprotnika, ki je zadnji udaril akterja) in *AI SpottedSoldier* (akterji uporabljajo komponento *PawnSensing*, tu se shrani referenca nasprotnika, ki je zadnji sprožil dogodek *OnSeePawn*). Če je cena ugodnejša od cene, shranjene v *AI CurrentPlanCost*, spremeni *AI TargetSoldier* in *AI CurrentPlanCost*. Ceno načrta za pokončanje vojaka iz *AI SoldierDealingDamage* ustrezno zmanjša in s tem spodbudi akterja, da se hitro odzove na napade, kar seveda počne tudi funkcija *CallNextGOAPAction*. Da igra ne bi bila pretežka, pred posodobitvijo *AI TargetSoldier* zmeraj preveri, koliko akterjev cilja istega nasprotnika. Če je število takih akterjev preveliko, ostane *AI TargetSoldier* nespremenjen.

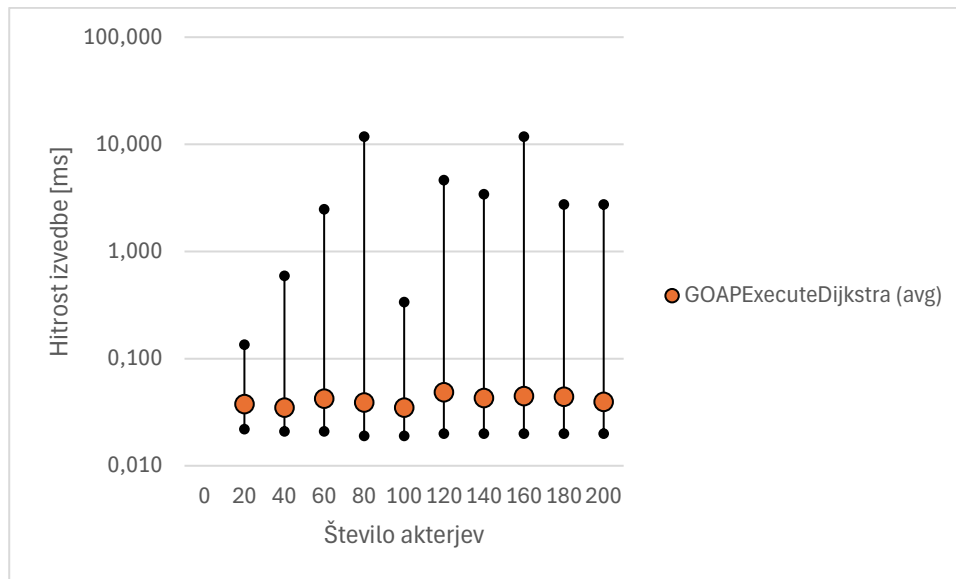
4 REZULTATI

V tem poglavju podrobno testiramo delovanje igre, pri čemer se osredotočimo na število sličic na sekundo (angl. *Frames per Second*, *FPS*). Analiza testiranja vključuje pregled, kako različni načini zagona igre vplivajo na tekoče delovanje in igralno izkušnjo. Med samim igranjem akterji igralcu dajejo vtis inteligence in zanj predstavljajo izziv, posebno, če ga hkrati napada več nasprotnikov. Dijkstrin algoritem smo pognali iz vsakega možnega začetnega vozlišča v grafu, pri čemer je algoritem vedno deloval hitro in stabilno ter vračal predvidene vrednosti. Kljub temu, da v naši implementaciji algoritma GOAP posamezen akter ne upošteva ostalih akterjev, igralcu dajejo občutek skupinskega delovanja. Tudi število sličic na sekundo, ki je sicer zaradi kompleksnosti naše scene dokaj nizko, se zaradi 80 inteligentnih akterjev, ki smo jih postavili na nivo, bistveno ne poslabša. Seveda pa igra ni brez pomanjkljivosti, ki so povezane predvsem z uporabo navigacijske mreže. Akterji na primer med navigacijo včasih naletijo na oviro, ki leži na navigacijski mreži, a je previsoka, da bi stopili čeznjo. Zato skušajo teči naprej, vendar jih ovira blokira in obstanejo na mestu. Rešijo se šele takrat, ko se njihova izračunana pot spremeni. Največja nerešena težava pa je, da Unreal omejuje število akterjev, ki lahko hkrati uporabljajo *DetourCrowdAIController* [37]. Žal imamo zato nekaj akterjev, ki tekom igre stojijo na mestu, sicer pa pravilno izvajajo algoritem GOAP in vse akcije, pri katerih se akter ne premika po sceni. Pri takšnih akterjih navigacija po nivoju ne porablja kapacitet računalnika, kar najbrž nekoliko izkrivi našo predstavo vpliva števila akterjev na FPS morebitne naslednje različice igre z odpravljenjo težavo.

Hitrost izvajanja odsekov kode v razredu *CPP_GoapPlanner* smo merili z uporabo Unrealove metode *FPlatformTime::Seconds*. Na začetku odseka smo vrnjeno vrednost shranili v spremenljivko *startTime*, na koncu pa v *endTime*. Seštevek *endTime* – *startTime* smo nato dodali v seznam. Na koncu smo izračunali povprečje vseh elementov seznama.

Inicializacija algoritma GOAP na našem računalniku (s procesorjem AMD Ryzen 5 5600X, grafično kartico NVIDIA GeForce RTX 3060 Ti in 16GB pomnilnika tipa DDR4) traja povprečno 0,876ms. Dijkstrin algoritem se izvaja na grafu z 2048 vozlišči. Od tega iz 1152 vozlišč izhajajo povezave naprej do sosednjih vozlišč. Poleg akcije *ActionDoNothing*, ki ne povezuje nobenega vozlišča v grafu, smo definirali 11 akcij. Ob upoštevanju definiranih pogojev in rezultatov posameznih akcij se ob inicializaciji v grafu ustvari 2336 povezav. Torej ima vsako vozlišče, iz katerega izhajajo povezave, v povprečju $2336 / 1152 \approx 2$ sosedu, ki ju kasneje ob klicu metode *GOAPExecuteDijkstra* preiskuje Dijkstrin algoritem.

Pri 60 akterjih izvedba metode *GOAPExecuteDijkstra* v povprečju traja 0,041ms. Ker se metoda izvaja na glavni niti, je povprečna hitrost izvedbe s povečevanjem števila akterjev praktično konstantna, kar je razvidno iz grafikona 1. Posledica je, da med izvajanjem zamrzne delovanje igre in zato še bolj vpliva na FPS. Med izvedbo Dijkstrinega algoritma se povprečno obiše 34,7 vozlišč. Pri tem je po izvedbi operacij, ki spreminjajo minimalno binarno kopico (metode *fixHeapAtIndex*, *insertNewNode* in *deleteRootNode*) v prednostni vrsti povprečno 7,1 vozlišč. Najvišje zabeleženo število vozlišč v vrsti je bilo 20, zato je čas izvajanja operacij nad kopico manjši od mikrosekunde. Med posameznim preiskovanjem grafa z Dijkstrinim algoritmom se v prednostni vrsti metoda *insertNewNode* povprečno kliče 33,7-krat, metodi *getMinDistanceNode* in *deleteRootNode* 34,7-krat, metoda *fixHeapAtIndex* pa 5,3-krat.

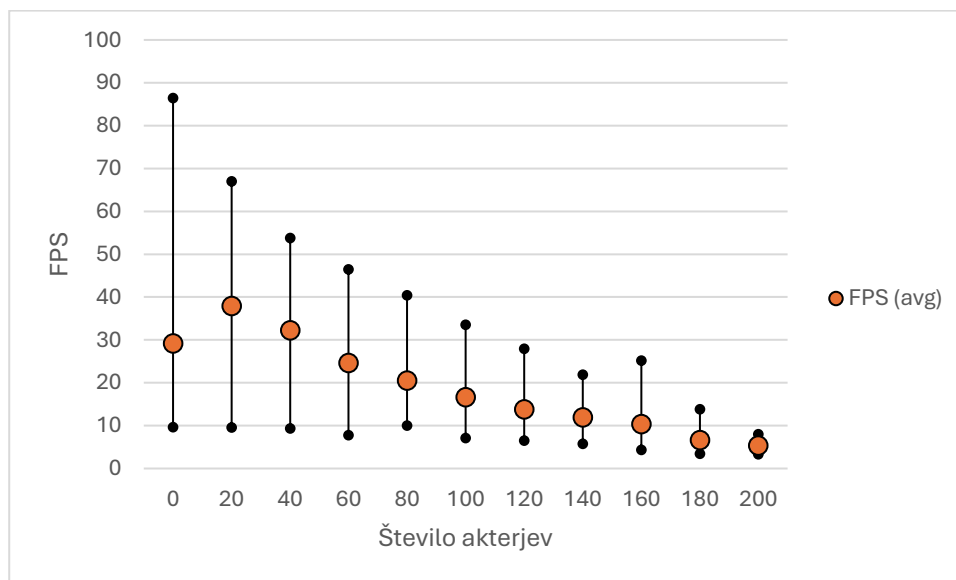


Grafikon 1: Hitrost izvedbe GOAPExecuteDijkstra v odvisnosti od števila akterjev

Metoda *GOAPGetKillPlayerCost* traja povprečno 0,105ms, kar je več kot dvakrat toliko, kot pri metodi *GOAPExecuteDijkstra*. Do tega pride, ker je naš trenutni graf dokaj slabo povezan. Ker so cilj lahko vsa vozlišča, kjer je parameter *playerDead* enak 1, metoda rekurzivno poišče $2^{11-1} = 1024$ potencialnih ciljev, med katerimi izbere najboljšega. Ker Dijkstra algoritem povprečno obiše le 34,7 vozlišč, je navadno velika večina rekurzivno najdenih ciljev iz začetnega vozlišča nedosegljiva. Navadno se nahajajo izven povezane komponente grafa, v kateri se nahaja izhodiščno vozlišče, ki je lahko tudi izolirano. Zato bi bilo pri našem grafu najbolje, da bi cilje vrednotili kar med izvajanjem Dijkstrinega algoritma.

Za oceno igralnosti je najbolj pomemben podatek, koliko sličic na sekundo dosega igra na našem računalniku. Število sličic je težko izmeriti, saj je odvisno od igralčevega položaja na sceni in smeri gledanja (izris dinamičnih luči, kompleksnih senčnikov (angl. *shaders*) in podrobnih tekstur močno vpliva na FPS igre [38]) in števila klicev funkcije *GOAPExecuteDijkstra*, ki je pri naši implementaciji odvisno predvsem od razdalje med igralcem in posameznim akterjem. FPS smo merili tako, da smo postopoma povečevali število akterjev na sceni (za vsakega branitelja smo postavili enega oblegovalca), nato pa igro igrali vse do konca. Implementirali smo graf Blueprint, ki je za vsako izrisano

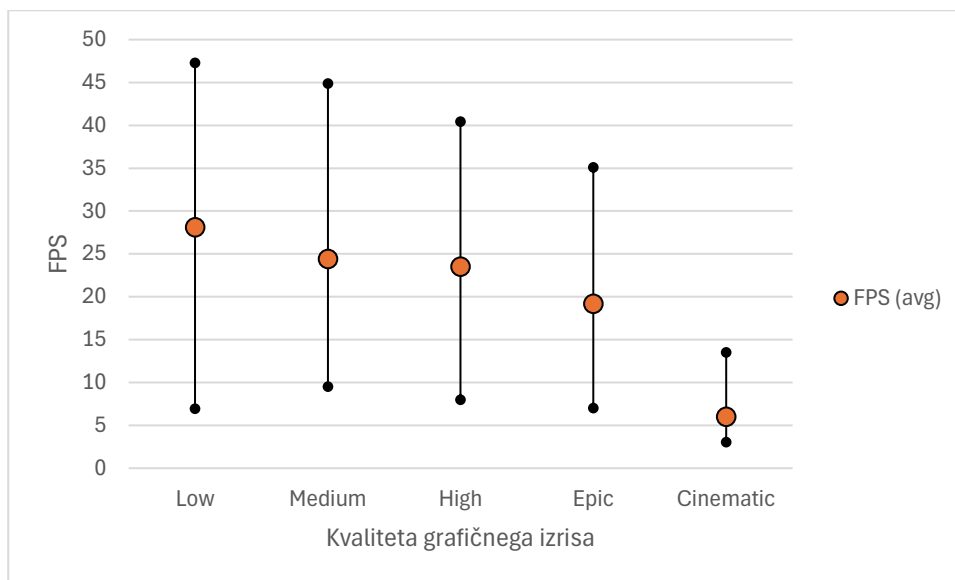
sliko v polje shranil hitrost izrisa. Ob pričetku igre se nalagajo viri in pri inteligentnih akterjih kličejo konstruktorji, v katerih se za vsakega posebej ustvari graf možnih stanj. To močno vpliva na FPS, zato smo z meritvami začeli takrat, ko se je stabiliziral. Kljub navedenim težavam pri merjenju grafikon 2 v skladu s pričakovanji odraža padanje FPS s povečevanjem števila akterjev. Izjema je prva povprečna vrednost na grafu (kadar na sceni ni nobenega akterja), saj smo se po sceni premikali na drugačen način, kot če bi igrali proti oblegovalcem. Seveda lahko igralec na prazni sceni pričakuje najvišje hitrosti, kar kaže tudi širok razpon med najnižjim in najvišjim zabeleženim FPS. Igro navadno igramo s 60 akterji, povprečen FPS je takrat 25FPS. Meja interaktivnosti je popolnoma subjektivna, nekateri jo postavljajo pri 15FPS. Povprečen FPS pade pod to mejo, kadar je na sceni 120 akterjev. Akterjev lik in opremljeno orožje so sicer sestavljeni iz 87.280 trikotnikov, kar Unrealov sistem *Nanite* tekom igre inteligentno prilagaja. Prav tako se med igranjem večina akterjev nahaja izven pogleda igralca, zato je vpliv izrisovanja 200 likov na FPS zelo majhen. Bistveno večji vpliv imajo drevesa in kompleksni senčniki. Pri testiranju smo uporabljali zgolj najvišje nastavitve (*Epic*).



Grafikon 2: FPS v odvisnosti od števila akterjev

Seveda lahko igralec število sličic na sekundo močno izboljša, če zniža kvaliteto grafičnega izrisa. Na grafikonu 3 je prikazano spreminjanje povprečnega FPS v

odvisnosti od nastavitev (*Engine Scalability Settings*). Merili smo ga pri 60 akterjih s prej opisanimi metodami. Pri tem smo spreminjali splošne nastavitve. Ker nastavitev *Cinematic* ni primerna za igranje, smo med razvojem uporabljali samo nastavitve *Epic*.



Grafikon 3: FPS v odvisnosti od kvalitete grafičnega izrisa

5 SKLEP

V diplomskem delu smo razvili in analizirali tretjeosebno računalniško igro z inteligentnimi akterji, postavljeno v utrjeno srednjeveško naselbino. Pri opisu teoretičnega ozadja smo podrobno predstavili razvoj igre v pogonu Unreal Engine 5.3, pri čemer smo uporabili programski jezik Blueprint, ki smo ga razširili z lastno funkcionalnostjo, implementirano v programskem jeziku C++. Prav tako smo predstavili Unrealov vtičnik za proceduralno postavljanje gradnikov in z algoritmom GOAP uspešno ustvarili inteligentne akterje, ki dajejo vtis delovanja v skupini.

Ugotovili smo, da je naša implementacija algoritma GOAP enostavno razširljiva in uporabna tudi v drugih projektih. Algoritem bi lahko v prihodnje nadgradili tako, da bi imeli akterji več ciljev, ki bi jih skušali doseči. Pri merjenju časa izvedbe posameznih metod smo našli metodo, ki porabi več časa, kot je potrebno in predlagali možno izboljšavo. Med testiranjem smo posebno ob velikem številu akterjev odkrili težave, ki bi jih bilo treba odpraviti.

Akterjem bi lahko dodali še več akcij, s čimer bi dosegli bolj zanimivo igro. Videz likov bi lahko spremenili z uporabo orodja *MetaHuman* in v igro dodali dialog. Prav tako bi lahko dodali večje število misij, orožij in animacij ter razvili kompleksnejši princip delovanja igre. Graf PCG bi lahko razširili, da bi bil ves teren pokrit z ustreznim rastlinjem. Na ta način bi igra za igralca postala še bolj zanimiva.

VIRI IN LITERATURA

- [1] Arora, K. *The Gaming Industry: A Behemoth With Unprecedented Global Reach*. 17. 11. 2023. [Elektronski] Dostopno na: <https://www.forbes.com/sites/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach/> [3. 6. 2024].
- [2] History. *Video Game History*. 1. 9. 2017. [Elektronski] Dostopno na: <https://www.history.com/topics/inventions/history-of-video-games> [3. 6. 2024].
- [3] Balasubramanian, K. *Game Engines: All You Need to Know*. 15. 9. 2023. [Elektronski] Dostopno na: <https://www.gameopedia.com/game-engines-all-you-need-to-know-about/> [3. 6. 2024].
- [4] Borowiec, S. *AlphaGo seals 4-1 victory over Go grandmaster Lee Sedol*. 15. 3. 2016. [Elektronski] Dostopno na: <https://www.theguardian.com/technology/2016/mar/15/googles-alphago-seals-4-1-victory-over-grandmaster-lee-sedol> [3. 6. 2024].
- [5] Sizer, B. *The Total Beginner's Guide to Game AI*. 13. 8. 2018. [Elektronski] Dostopno na: <https://www.gamedev.net/tutorials/programming/artificial-intelligence/the-total-beginners-guide-to-game-ai-r4942/> [3. 6. 2024].
- [6] Rabin, S. *Game AI Pro: Collected Wisdom of Game AI Professionals*. Boca Raton: CRC Press, 2014.
- [7] Millington, I., *Artificial Intelligence for Games*. San Francisco: Morgan Kaufman Publishers, 2006.
- [8] Nez. *AI (FSM, Behavior Tree, GOAP, Utility AI)*. 26. 9. 2019. [Elektronski] Dostopno na: <https://anshuman-kumar.gitbook.io/nez-doc/ai-fsm-behavior-tree-goap-utility-ai> [3. 6. 2024].
- [9] Epic Games Inc. *Behavior Tree User Guide*. [Elektronski] Dostopno na: https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---user-guide?application_version=5.0 [3. 6. 2024].

- [10] Orkin, J. *Three States and a Plan: The A.I. of F.E.A.R.* Cambridge: MIT Media Lab, 2006.
- [11] Epic Games Inc. *Unreal Engine*. [Elektronski] Dostopno na: <https://www.unrealengine.com/en-US> [30. 7. 2024].
- [12] Blender Foundation. *Time to Interact*. [Elektronski] Dostopno na: <https://www.blender.org/download/releases/3-6/> [30. 7. 2024].
- [13] Adobe Systems Incorporated. *Mixamo*. 2024. [Elektronski] Dostopno na: <https://www.mixamo.com> [11. 6. 2024].
- [14] Terribilis Studio Inc. *Mixamo Converter*. 29. 10. 2022. [Elektronski] Dostopno na: <https://terribilisstudio.fr/?section=MC> [11. 6. 2024].
- [15] QuadSpinner. *Gaea 2.0 Pre-Orders are now available!*. [Elektronski] Dostopno na: <https://quadspinner.com/> [30. 7. 2024].
- [16] Microsoft. *GitHub Copilot woven into Visual Studio*. [Elektronski] Dostopno na: <https://visualstudio.microsoft.com/> [30. 7. 2024].
- [17] Epic Games Inc. *Features*. [Elektronski] Dostopno na: <https://www.unrealengine.com/en-US/features> [4. 6. 2024].
- [18] Epic Games Inc. *Licensing*. [Elektronski] Dostopno na: <https://www.unrealengine.com/en-US/license> [4. 6. 2024].
- [19] Epic Games Inc. *MetaHuman Creator*. [Elektronski] Dostopno na: <https://dev.epicgames.com/documentation/en-us/metahuman/metahuman-creator-overview> [4. 6. 2024].
- [20] Epic Games Inc. *Blueprint Overview*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.26/en-US/ProgrammingAndScripting/Blueprints/Overview/> [11. 6. 2024].
- [21] Blender Foundation. *The Freedom to Create*. [Elektronski] Dostopno na: <https://www.blender.org/about/> [4. 6. 2024].

- [22] Epic Games Inc. *Niagara Overview*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/RenderingAndGraphics/Niagara/Overview/> [7. 6. 2024].
- [23] Kenny. *Procedural Generation: An Overview*. 19. 2. 2021. [Elektronski] Dostopno na: <https://kentpawson123.medium.com/procedural-generation-an-overview-1b054a0f8d41> [7. 6. 2024].
- [24] Epic Games Inc. *Procedural Content Generation Overview*. [Elektronski] Dostopno na: https://dev.epicgames.com/documentation/en-us/unreal-engine/procedural-content-generation-overview?application_version=5.2 [7. 6. 2024].
- [25] Grafenauer, B. *Zgodovina slovenskega naroda. Zv. 1, Od naselitve do uveljavljanja frankovskega reda (z uvodnim pregledom zgodovine slovenskega ozemlja do naselitve alpskih Slovanov)*, druga izdaja. Ljubljana: Državna založba Slovenije, 1964.
- [26] Epic Games Inc. *UFunctions*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/GameplayArchitecture/Functions/> [9. 6. 2024].
- [27] Epic Games Inc. *Properties*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/GameplayArchitecture/Properties/> [9. 6. 2024].
- [28] Epic Games Inc. *Coding Standard*. [Elektronski] Dostopno na: <https://dev.epicgames.com/documentation/en-us/unreal-engine/epic-cplusplus-coding-standard-for-unreal-engine> [9. 6. 2024].
- [29] Epic Games Inc. *Actor Lifecycle*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/ProgrammingWithCPP/UnrealArchitecture/Actors/ActorLifecycle/> [9. 6. 2024].
- [30] Venkat, R. J. *PATH FINDING: Dijkstra's Algorithm*. Terre Haute: Indiana State University, 2014.

- [31] Mehlhorn, K., Sanders, P. *Algorithms and Data Structures: The Basic Toolbox*. Heidelberg: Springer, 2008.
- [32] Sedgewick, R., Wayne, P. *Algorithms*, četrta izdaja. Boston: Pearson Education, 2011.
- [33] Epic Games Inc. *Animation*. [Elektronski] Dostopno na: <https://www.unrealengine.com/en-US/uses/animation> [11. 6. 2024].
- [34] Hicar, S. *Motion Capture: Movement Digitization Process*. [Elektronski] Dostopno na: <https://www.britannica.com/technology/motion-capture> [11. 6. 2024].
- [35] Epic Games Inc. *Using Animation Montages*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/AnimatingObjects/SkeletalMeshAnimation/AnimMontage/UserGuide/> [11. 6. 2024].
- [36] Epic Games Inc. *Basic Navigation*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/4.27/en-US/InteractiveExperiences/ArtificialIntelligence/NavigationSystem/BasicNavigation/> [20. 6. 2024].
- [37] Epic Games Inc. *Using Avoidance With the Navigation System*. [Elektronski] Dostopno na: <https://dev.epicgames.com/documentation/en-us/unreal-engine/using-avoidance-with-the-navigation-system-in-unreal-engine> [31. 7. 2024].
- [38] Epic Games Inc. *GPU Performance and Profiling*. [Elektronski] Dostopno na: <https://docs.unrealengine.com/udk/Three/GPUProfilingHome.html> [20. 6. 2024].